



GUÍA DE ESTUDIO Y CURSO COMPLETO

Guía de estudio ISTQB Foundation Level (CTFL)

Fecha	30 de agosto de 2026	Periodo	Temario oficial v4.0 · curso completo, guía y tres exámenes
Dirigido a	Candidatas y candidatos a Grupo Mazi	Lugar	Santiago de Querétaro, Qro.
Folio	GM-GEN-20260830-SKN		

ANTES DE EMPEZAR: QUÉ ES ESTO Y CÓMO SE USA

Si estás leyendo esto es porque quieres entrar a Grupo Mazi, o porque alguien de aquí pensó que valía la pena que lo leyeras. En los dos casos te conviene saber una cosa desde el primer renglón: **esta guía está hecha para que apruebes el examen sin comprar ningún otro material**. No es un resumen que te manda a leer el temario oficial. Es el temario, explicado, con ejemplos y con exámenes para que compruebes si lo entendiste.

Va escrita como si te lo estuviera contando en una mesa, no como un manual. Eso no significa que esté aguado: significa que cada concepto viene con el ejemplo que lo hace obvio, y después con el caso técnico donde se complica. Si un párrafo te suena a relleno, es que lo escribí mal.

Qué es la certificación

El **ISTQB Certified Tester Foundation Level** —lo verás abreviado como **CTFL**— es la certificación de entrada del International Software Testing Qualifications Board. Es la más reconocida del mundo en pruebas de software: la piden en ofertas de trabajo de bancos, aseguradoras, consultoras y fábricas de software de cualquier país.

Lo que acredita no es que sepas usar una herramienta. Acredita que **hablas el idioma de la calidad**: que cuando alguien dice «regresión», «partición de equivalencia» o «criterio de salida», tú entiendes lo mismo que la persona de enfrente. En un equipo donde cada quien usa las palabras a su manera, eso vale más de lo que parece.



Qué NO es

- No es un curso de programación. Puedes certificarte sin escribir una línea de código, aunque saber programar te va a servir después.
- No es un curso de una herramienta concreta. No enseña Selenium, ni Cypress, ni JMeter. Enseña **qué** probar y **por qué**, no con qué botón.
- No te vuelve tester por sí sola. Te da el vocabulario y el marco. La experiencia la das tú.

EL EXAMEN: EXACTAMENTE QUÉ TE VAN A PEDIR

Conviene que esto lo tengas claro desde el principio, porque cambia cómo estudias.

Formato	40 preguntas de opción múltiple
Duración	60 minutos
Puntaje para aprobar	65 por ciento, es decir 26 de 40 puntos
Idioma	hay examen en español
Tiempo extra	25 % adicional si el examen no está en tu lengua materna
Material	examen a libro cerrado

Ojo con el puntaje. No todas las preguntas valen un punto. Algunas valen dos o tres, y suelen ser las que piden aplicar una técnica —calcular casos de prueba, por ejemplo—. Por eso las técnicas del capítulo 4 pesan más de lo que sugiere el número de preguntas.

Los niveles cognitivos, que sí importan

Cada objetivo de aprendizaje del temario trae una etiqueta **K1**, **K2** o **K3**. Suena a burocracia y no lo es: te dice **qué tipo de pregunta** te pueden hacer sobre ese punto.

Nivel	Qué se te pide	Cómo suena la pregunta
K1 · Recordar	Reconocer o recordar un término	«¿Cuál de estas es la definición de...?»
K2 · Comprender	Explicar, comparar, clasificar	«¿Cuál es la diferencia entre...?»



Nivel	Qué se te pide	Cómo suena la pregunta
K3 · Aplicar	Usar una técnica en un caso concreto	«Dado este código, ¿cuántos casos hacen falta para...?»

Si un tema es K1, memorizarlo alcanza. Si es K3, **tienes que haberlo hecho con las manos** al menos tres o cuatro veces. Los K3 son casi todos del capítulo 4, que es el de las técnicas de diseño de pruebas.

Cuánto pesa cada capítulo

Capítulo	Tema	Peso aproximado
1	Fundamentos de las pruebas	20 %
2	Pruebas a lo largo del ciclo de vida	20 %
3	Pruebas estáticas	10 %
4	Análisis y diseño de pruebas	30 %
5	Gestión de la actividad de prueba	12 %
6	Herramientas de prueba	8 %

Traducido: **el capítulo 4 vale casi un tercio del examen** y es donde más gente falla, porque es el único donde hay que calcular. Si vas corto de tiempo, ahí es donde inviertes.



ÍNDICE

Los números son de la hoja impresa, no del archivo.

Apartado	Página
Antes de empezar: qué es esto y cómo se usa	1
El examen: exactamente qué te van a pedir	2
I. Fundamentos de las pruebas	5
II. Pruebas a lo largo del ciclo de vida	9
III. Pruebas estáticas	12
IV. Análisis y diseño de pruebas	15
V. Gestión de la actividad de prueba	22
VI. Herramientas de prueba	26
Guía de estudio: tres semanas, una hora al día	27
Examen 1 · Fundamentos, ciclo de vida y pruebas estáticas	31
Examen 2 · Técnicas de diseño	36
Examen 3 · Simulacro completo, cronometrado	42
Hoja de respuestas 1 · Fundamentos, ciclo de vida y pruebas estáticas	53
Hoja de respuestas 2 · Técnicas de diseño	56
Hoja de respuestas 3 · Simulacro completo	59
Glosario · los términos que sí caen	64
Dónde presentar el examen y quién da el curso	71
Para leer después · enlaces comprobados	80



I. FUNDAMENTOS DE LAS PRUEBAS

Qué es probar, en serio

Mucha gente cree que probar es «buscar errores». Se queda corto. Probar es **evaluar que algo cumple lo que se esperaba de él, y encontrar dónde no lo cumple**. Eso incluye buscar defectos, sí, pero también incluye dar confianza: cuando pruebas y todo pasa, has producido información valiosa aunque no hayas encontrado nada.

Y ahí está la primera idea que hay que tener firme para el examen: **probar produce información para decidir**. No es un trámite de calidad, no es firmar un papel. Alguien va a decidir si esto sale a producción, y tu trabajo es que lo decida sabiendo.

Ejemplo simple: pruebas una calculadora. Sumas $2 + 2$ y da 4. No encontraste nada, y aun así ahora sabes algo que antes no sabías.

Ejemplo técnico: pruebas un servicio de cobros con 300 casos. Los 300 pasan. El equipo despliega el viernes con esa información. Si no hubieras probado, el despliegue del viernes habría sido una apuesta.

Objetivos típicos de las pruebas

- Evaluar productos de trabajo: requisitos, historias de usuario, diseño, código.
- Provocar fallos y encontrar defectos.
- Asegurar la cobertura requerida del objeto de prueba.
- Reducir el riesgo de que quede calidad insuficiente.
- Verificar si se cumplen los requisitos especificados.
- Comprobar que el objeto cumple requisitos contractuales, legales o normativos.
- Dar información a las personas que deciden.
- Generar confianza en la calidad del objeto.
- Validar que el producto está completo y funciona **como esperan las personas usuarias**.



Fíjate en los dos últimos: **verificar** es comprobar contra la especificación; **validar** es comprobar contra la necesidad real. Un producto puede pasar la verificación entera y fallar la validación, porque hicimos exactamente lo que decía el documento y el documento estaba mal.

Probar y depurar no son lo mismo

Es una pregunta clásica de examen.

	Probar	Depurar
Quién	Normalmente quien prueba	Normalmente quien desarrolla
Qué hace	Provoca fallos causados por defectos	Encuentra la causa del defecto, la analiza y la corrige
Resultado	Un reporte de defecto	Un cambio en el código

Después de depurar, alguien vuelve a probar: eso se llama **prueba de confirmación**, y lo vemos en el capítulo 2.

Error, defecto, fallo y causa raíz

Esta cadena entra en el examen casi seguro, y la gente la confunde. Va con un ejemplo que no se olvida.

- 1. Error** (equivocación): una persona se equivoca. El programador teclea `>` donde iba `>=`.
- 2. Defecto** (falla, bug): lo que queda en el producto por culpa de ese error. En el código hay un `>` que no debía estar.
- 3. Fallo**: lo que se ve cuando el defecto se ejecuta. El sistema deja comprar a un cliente de 17 años porque la validación de mayoría de edad está mal.
- 4. Causa raíz**: por qué ocurrió el error. Por ejemplo: el requisito decía «mayores de edad» sin definir si 18 entra o no, y nadie preguntó.

Tres cosas que hay que tener claras:



- **Un defecto no siempre produce un fallo.** Puede estar en una rama de código que nadie ejecuta nunca.
- **No todo fallo viene de un defecto.** Puede ser el entorno: radiación, contaminación, campos magnéticos, una configuración de red.
- **Atacar la causa raíz evita defectos futuros.** Corregir el > arregla este caso; aclarar el requisito evita los otros veinte.

Los siete principios de las pruebas

Son de memoria obligatoria, y de los que más se preguntan. Te los pongo con la traducción a lo que significan en el día a día.

1 · Las pruebas muestran la presencia de defectos, no su ausencia. Puedes probar mil casos y encontrar cero defectos. Eso no demuestra que no haya: demuestra que en esos mil casos no aparecieron. No se puede probar «que no hay bugs».

2 · Las pruebas exhaustivas son imposibles. Salvo en casos triviales. Un formulario con tres campos de texto libre ya tiene infinitas combinaciones. Por eso existen las técnicas del capítulo 4: para escoger bien el subconjunto que sí se va a probar.

3 · Las pruebas tempranas ahorran tiempo y dinero. Es el famoso *shift left*. Un defecto encontrado en el requisito cuesta corregirlo una fracción de lo que cuesta en producción. Y no es sólo dinero: en producción también cuesta reputación.

4 · Los defectos se agrupan. Un porcentaje pequeño de módulos concentra la mayoría de los defectos. Si un módulo ya dio problemas, ahí es donde hay que seguir buscando. Es la base de las pruebas basadas en riesgo.

5 · Las pruebas se desgastan (paradoja del pesticida). Si corres siempre las mismas pruebas, dejan de encontrar cosas nuevas — igual que un pesticida al que los insectos se hacen resistentes. Hay que revisar y renovar los casos. Aunque ojo: para **regresión** la repetición sí es el objetivo, y ahí el desgaste no es un problema.

6 · Las pruebas dependen del contexto. No se prueba igual un videojuego que un marcapasos. El contexto decide el nivel de rigor, las técnicas y la cobertura.



7 · La ausencia de defectos es una falacia. Puedes entregar un sistema sin defectos conocidos y aun así fracasar, porque no era lo que la gente necesitaba. Cumplir la especificación no garantiza el éxito.

Truco para el examen: casi todas las preguntas sobre principios te dan una situación y preguntan qué principio ilustra. Practica al revés: por cada principio, invéntate una situación de tu propio trabajo.

Las actividades de prueba

El proceso de prueba no es una fase: es un conjunto de actividades que se solapan.

- 1. Planificación:** definir objetivos y cómo alcanzarlos en este contexto.
- 2. Monitorización y control:** comparar el avance real con el plan y actuar.
- 3. Análisis:** decidir **qué** probar. Se estudian los productos de trabajo —la base de prueba— y se identifican condiciones de prueba.
- 4. Diseño:** decidir **cómo** probar. Las condiciones se convierten en casos de prueba y conjuntos de casos, aplicando técnicas.
- 5. Implementación:** preparar lo que hace falta para ejecutar —datos, entornos, guiones, procedimientos—.
- 6. Ejecución:** correr las pruebas y comparar resultado real contra esperado.
- 7. Compleción:** cerrar, archivar, reportar lecciones aprendidas.

Una confusión muy repetida: **análisis** es *qué* probar; **diseño** es *cómo*. Si la pregunta dice «identificar condiciones de prueba», es análisis. Si dice «derivar casos de prueba», es diseño.

Trazabilidad: por qué hay que poder seguir el hilo

Cada elemento de prueba debe poder rastrearse hasta su base: el requisito, la historia, el riesgo. Sirve para tres cosas muy prácticas:



- Saber qué cobertura tienes de verdad.
- Analizar el impacto cuando algo cambia.
- Explicarle a alguien que no es técnico qué se probó y qué falta.

Habilidades: no todo es técnica

El temario lo dice y suele salir alguna pregunta. Un tester necesita conocimiento del dominio, pensamiento crítico y analítico, atención al detalle y **comunicación**. Esa última es la que decide si te hacen caso.

Psicología de las pruebas: el lado incómodo

Encontrar defectos se puede leer como criticar el trabajo de alguien. Por eso el temario habla en serio de cómo se comunican los hallazgos:

- Empieza por colaborar, no por acusar. El objetivo común es un mejor producto.
- Reporta **hechos neutrales y objetivos**, sin juicios sobre la persona.
- Explica el beneficio de conocer el problema.
- Comprueba que la otra persona entendió, y entiende tú por qué se hizo así.

Y el **sesgo de confirmación**: cuesta aceptar información que contradice lo que uno cree. Por eso alguien distinto de quien escribió el código encuentra cosas que el autor no ve. No es falta de talento: es que uno no ve lo que acaba de hacer.

La independencia tiene grados. Que pruebe el propio autor cuesta menos y ve menos; que pruebe otra persona del equipo ve más; que pruebe un equipo externo ve todavía más pero cuesta y aleja el conocimiento del producto. No hay una respuesta correcta: hay un equilibrio según el contexto.

II. PRUEBAS A LO LARGO DEL CICLO DE VIDA

Cómo cambian las pruebas según el modelo de desarrollo



El modelo de desarrollo no cambia **qué** es probar, pero cambia **cuándo** y **cómo**.

- En modelos **secuenciales** (cascada, V), cada fase termina antes de que empiece la siguiente. Las pruebas de ejecución llegan tarde, aunque las estáticas pueden empezar desde el principio.
- En modelos **iterativos e incrementales**, el producto crece por trozos. Se prueba cada incremento y además hay que asegurar que lo nuevo no rompió lo viejo — de ahí que la **regresión** crezca en importancia.
- En **ágil**, las pruebas están dentro del equipo y ocurren continuamente. Aparecen enfoques colaborativos: criterios de aceptación escritos entre todos, desarrollo guiado por pruebas, integración continua.

Dos ideas que se preguntan mucho:

Shift left. Probar lo antes posible. No significa sólo «adelantar la fase de pruebas»: significa revisar requisitos, escribir pruebas antes que el código, y que quien desarrolla haga pruebas unitarias y de componente. Lo caro no es probar temprano, es corregir tarde.

Retrospectivas. Al terminar un ciclo, el equipo revisa qué funcionó y qué no, y se compromete a cambios concretos. Es donde el proceso de prueba mejora de verdad.

Niveles de prueba

Son grupos de actividades organizados por el objeto que se prueba. **No son fases**: en un proyecto pueden solaparse.

Nivel	Qué se prueba	Quién suele hacerlo	Base típica
Componente (unitarias)	Módulos y funciones por separado	Desarrollo	Diseño detallado, código
Integración de componentes	Las interfaces entre módulos	Desarrollo	Diseño de software
Sistema	El sistema completo	Equipo de pruebas	Requisitos, casos de uso



Nivel	Qué se prueba	Quién suele hacerlo	Base típica
Integración de sistemas	Interfaces con sistemas externos	Equipo de pruebas	Arquitectura, interfaces
Aceptación	Que sirve para lo que se pidió	Cliente, negocio, usuario	Necesidades, procesos, contratos

Los cuatro niveles sobre una misma app de tienda, para que se vea la escalera:

- **Componente:** la función que calcula el IVA devuelve 16 pesos sobre 100.
- **Integración:** el carrito llama bien a esa función y guarda el total.
- **Sistema:** se puede completar una compra de principio a fin.
- **Aceptación:** el dueño de la tienda confirma que el flujo es el que necesitaba.

Tipos de prueba de aceptación que hay que conocer:

- **De usuario:** quien va a usarlo comprueba que sirve para su trabajo.
- **Operativa:** respaldos, restauración, seguridad, migración, desinstalación.
- **Contractual y normativa:** lo que exige el contrato o la ley.
- **Alfa:** en las instalaciones de quien desarrolla, con usuarios externos.
- **Beta:** en el sitio de las personas usuarias, con su propio equipo.

Tipos de prueba

Un tipo agrupa pruebas por **característica de calidad** que evalúan.

- **Funcionales:** qué hace el sistema. ¿Calcula bien el descuento?
- **No funcionales:** cómo lo hace. Rendimiento, usabilidad, seguridad, fiabilidad, portabilidad, mantenibilidad, compatibilidad.
- **De caja negra:** se basan en la especificación, sin mirar el interior.
- **De caja blanca:** se basan en la estructura interna, el código o la arquitectura.



Error frecuentísimo: creer que «caja blanca» es lo mismo que «unitaria», o que «no funcional» es lo mismo que «rendimiento». Son ejes distintos: puedes hacer una prueba **no funcional de caja blanca** a nivel de **sistema**. Los tres ejes —nivel, tipo y técnica— se combinan libremente.

Confirmación y regresión

Se preguntan juntas y se confunden.

- **Prueba de confirmación:** se corrigió un defecto, se vuelve a ejecutar el caso que fallaba para comprobar que ya no falla.
- **Prueba de regresión:** se comprueba que el cambio **no rompió otra cosa** que antes funcionaba.

La regresión es la primera candidata a automatizarse, porque se repite mucho y es aburridísima a mano. Y aquí el principio del desgaste no aplica: en regresión, repetir es justamente el punto.

Pruebas de mantenimiento

Cuando el sistema ya está en producción y se le hace un cambio —corrección, mejora, migración, retirada de una función—, hay que probar el cambio y hacer regresión de lo que sigue vivo.

Lo que decide cuánto probar es el **análisis de impacto**: qué partes toca el cambio. Y ahí duele la falta de trazabilidad: sin ella, el análisis de impacto es una adivinanza.

Disparadores típicos de mantenimiento: modificaciones, actualizaciones del entorno operativo, migración de datos, y retirada del sistema.

III. PRUEBAS ESTÁTICAS



Probar sin ejecutar

Las pruebas estáticas examinan productos de trabajo **sin ejecutarlos**. Se puede probar estáticamente casi cualquier cosa: requisitos, historias de usuario, criterios de aceptación, código, arquitectura, contratos, manuales, casos de prueba.

Dos formas:

- **Revisiones:** personas leyendo. Manual.
- **Análisis estático:** herramientas que revisan el código o los modelos sin ejecutarlos.

Por qué valen tanto

Porque encuentran **defectos**, no fallos, y los encuentran temprano.

- Detectan cosas que la ejecución no puede ver: requisitos ambiguos, contradicciones, omisiones, código muerto, dependencias mal declaradas.
- Un defecto en un requisito, cazado al revisarlo, cuesta una fracción de lo que cuesta cuando ya está programado, probado y desplegado.
- Mejoran la comunicación del equipo: al revisar juntos, todos entienden lo mismo.

Ejemplo simple: en la revisión de un requisito alguien pregunta «¿los 18 años cumplidos entran o no?». Ese defecto nunca llegó a existir en el código.

Ejemplo técnico: el análisis estático marca una variable que se lee antes de asignarse en una rama poco frecuente. Esa rama no la ejecutaba ninguna prueba dinámica.

**Revisiones: los tipos que hay que distinguir**

Tipo	Formalidad	Quién lo lidera	Para qué sirve
Informal	Ninguna	Nadie en particular	Detectar defectos rápido, sin proceso
Guiada (walkthrough)	Baja	El autor	Evaluar calidad, formar, consensuar
Técnica	Media	Un moderador o el autor	Decisiones técnicas, alternativas
Inspección	Alta	Un moderador formado	Encontrar el máximo de defectos, medir el proceso

La **inspección** es la más formal: roles definidos, criterios de entrada y salida, métricas, y el autor **no** la lidera.

Roles en una revisión formal

- **Autor:** creó el producto y lo corrige después.
- **Gerencia:** decide que se hagan revisiones y da el tiempo.
- **Facilitador o moderador:** dirige la reunión, media, garantiza que sea efectiva y segura.
- **Líder de revisión:** planifica, decide quién participa, cuándo y dónde.
- **Revisor:** encuentra hallazgos. Puede ser experto del dominio, del negocio, o de otra especialidad.
- **Escriba o registrador:** anota los hallazgos.



El proceso, en orden

1. **Planificación:** qué se revisa, con qué criterios, quién participa.
2. **Inicio:** se reparte el material y se explica el objetivo.
3. **Revisión individual:** cada quien examina por su cuenta y anota.
4. **Comunicación y análisis:** se juntan los hallazgos, se decide cuáles son defectos y quién los corrige.
5. **Corrección y reporte:** el autor corrige; se comprueba si se cumplen los criterios de salida.

La regla que hace que una revisión funcione: se revisa el **producto**, no a la persona. Y los hallazgos se registran sin juicios. Una revisión que se vive como examen personal deja de producir hallazgos a la segunda vez.

Factores de éxito que suelen preguntarse: objetivos claros, tipo de revisión adecuado al contexto, trozos pequeños, retroalimentación rápida, tiempo suficiente para la preparación, apoyo de la gerencia, y hacerlo parte de la cultura del equipo.

IV. ANÁLISIS Y DISEÑO DE PRUEBAS

Éste es el capítulo que más pesa —cerca de un tercio del examen— y el único donde vas a tener que **calcular**. Si sólo lo lees, lo fallas. Haz los ejercicios con lápiz.

Las tres familias de técnicas



Familia	En qué se basa	Cuándo se usa
Caja negra	La especificación: requisitos, historias, casos de uso	Siempre que haya algo escrito que diga qué debe hacer
Caja blanca	La estructura interna: código, arquitectura, flujo	Cuando puedes ver el interior y quieres medir cobertura
Basadas en experiencia	El conocimiento de quien prueba	Cuando la especificación es pobre, o para complementar

Ninguna sustituye a las otras. En un proyecto real se combinan.

1 · Partición de equivalencia

La idea: si dos valores van a recibir el mismo trato del sistema, probar los dos no aporta más que probar uno. Se agrupan los valores en clases donde todos se comportan igual, y se prueba **un representante de cada clase**.

Ejemplo simple. Un campo acepta edades de 18 a 65.

Clase	Rango	Válida	Un valor
Menores	0 a 17	No	10
En rango	18 a 65	Sí	40
Mayores	66 en adelante	No	70

Con **tres** casos cubres las tres clases. Probar 19, 20, 21... no añade nada.

Cobertura: porcentaje de particiones cubiertas por al menos un caso. Si pruebas 40 y 70 pero no un menor de 18, tu cobertura es $2 \text{ de } 3 = 66 \%$.

Ejemplo técnico. Un cobro con descuento por volumen:



- 1 a 9 unidades: sin descuento
- 10 a 49: 10 %
- 50 o más: 20 %
- 0 o negativo: error

Cuatro particiones válidas o inválidas, cuatro casos: 5, 25, 80 y -3.

Trampa clásica del examen: te dan un enunciado con varias particiones y preguntan «cuántos casos como mínimo para cobertura del 100 %». La respuesta es el **número de particiones**, no el número de valores.

2 · Análisis de valores límite

La idea: los defectos se acumulan **en los bordes**. El > que debía ser >= sólo se nota justo en el límite.

Se usa sobre particiones **ordenadas** (números, fechas). Hay dos formas:

- **De dos valores:** se prueban el límite y su vecino inmediato al otro lado.
- **De tres valores:** el límite, el anterior y el siguiente.

Ejemplo simple. Edad válida de 18 a 65.

Forma	Valores a probar
Dos valores	17, 18, 65, 66
Tres valores	17, 18, 19, 64, 65, 66

Ejemplo técnico. Un campo de texto acepta de 1 a 100 caracteres. Los límites no son sólo 1 y 100: también son 0 y 101. Y en sistemas reales conviene mirar los límites del **tipo de dato**: 255, 256, 65 535, y el desbordamiento de enteros.

Cobertura: porcentaje de valores límite ejecutados sobre el total de valores límite identificados.



Cómo no equivocarse: primero identifica particiones, después sus límites. Quien salta directo a los límites se olvida de particiones enteras.

3 · Tabla de decisión

La idea: cuando el resultado depende de **varias condiciones combinadas**, una tabla evita que se te olvide una combinación.

Ejemplo simple. Un descuento se aplica si el cliente es socio **y** compra más de 1 000 pesos.

Condiciones	C1	C2	C3	C4
¿Es socio?	Sí	Sí	No	No
¿Compra > 1000?	Sí	No	Sí	No
Acción: descuento	Sí	No	No	No

Con dos condiciones booleanas salen $2^2 = 4$ columnas. Con tres, 8. Con cuatro, 16 — y ahí ya conviene simplificarla.

Tabla colapsada. Si una condición no cambia el resultado en cierto caso, se marca con un guion. En el ejemplo, si no es socio da igual cuánto compre: las columnas C3 y C4 se funden en una sola con «-» en la segunda condición. Se pasa de 4 columnas a 3, y las tres siguen cubriendo todo.

Cobertura: porcentaje de columnas —reglas— ejecutadas. Cada columna es al menos un caso de prueba.

Ejemplo técnico. Aprobación de crédito con tres condiciones: ingresos suficientes, historial limpio, antigüedad mayor a un año. Ocho combinaciones, y el negocio dice que basta con que falle una para rechazar. La tabla colapsada queda en cuatro reglas y se ve de inmediato que la única aprobación es la combinación de tres síes.



4 · Prueba de transición de estados

La idea: algunos sistemas se comportan distinto según **en qué estado están**. Se dibuja el diagrama de estados y se prueban las transiciones.

Ejemplo simple. Una sesión de usuario.

Estado actual	Evento	Estado siguiente
Desconectado	Entrar con datos correctos	Conectado
Desconectado	Entrar con datos incorrectos	Desconectado
Conectado	Cerrar sesión	Desconectado
Conectado	Expirar el tiempo	Desconectado

Cuatro transiciones válidas, cuatro casos mínimos.

Tres niveles de cobertura que hay que distinguir:

- **Cobertura de todos los estados:** cada estado se visita al menos una vez. Es la más débil.
- **Cobertura de transiciones válidas** (0-switch): cada transición del diagrama se ejerce al menos una vez. Es la que se pide normalmente.
- **Cobertura de todas las transiciones:** incluye las **inválidas** — qué pasa si llega un evento que en ese estado no debería llegar. Es la más fuerte y la que encuentra los defectos raros.

Ejemplo técnico. Un cajero automático: en estado «esperando PIN», ¿qué pasa si llega el evento «retirar efectivo»? En un sistema bien hecho, nada. En uno mal hecho, ahí está el defecto.

5 · Técnicas de caja blanca

Se basan en la estructura interna. En Foundation Level entran dos, y hay que saber calcularlas.



Cobertura de sentencias. Porcentaje de líneas ejecutables que ejecutaron tus pruebas.

Cobertura de ramas (decisión). Porcentaje de resultados de decisión —cada **sí** y cada **no** de cada **if**— que se ejercieron.

El ejemplo que lo aclara todo. En las tablas de código de esta guía, el punto al principio de una línea es la sangría: quiere decir que esa línea va dentro del bloque de arriba.

Línea	Código
1	leer A
2	si A > 10 entonces
3	· imprimir "grande"
4	fin si
5	imprimir "listo"

Con **un solo caso**, A = 20:

- Ejecuta las líneas 1, 2, 3 y 5 → 4 de 4 sentencias ejecutables → **100 % de sentencias.**
- Del **if** sólo ejerció la salida verdadera → 1 de 2 ramas → **50 % de ramas.**

Con A = 20 y A = 5 tienes 100 % de sentencias y 100 % de ramas.

La regla que se pregunta seguro: el 100 % de cobertura de ramas **garantiza** el 100 % de sentencias. Al revés **no**. Con el ejemplo de arriba lo compruebas en diez segundos, y por eso conviene memorizarlo con el ejemplo y no con la frase.

Para qué sirve de verdad la caja blanca. No para sustituir a la caja negra, sino para **medir** cuánto de lo que existe tocaste. Si tus pruebas funcionales dejan el 40 % del código sin ejecutar, ahí hay algo que nadie está mirando: o son casos que faltan, o es código muerto.



6 · Técnicas basadas en la experiencia

Adivinación de errores (error guessing). Se usa la experiencia para imaginar dónde suele fallar: campos vacíos, cero, valores negativos, textos larguísimos, caracteres especiales, doble clic, red intermitente. Funciona muy bien y no es sistemática: por eso complementa, no sustituye.

Pruebas exploratorias. Se diseña, ejecuta y evalúa **al mismo tiempo**, guiándose por lo que se va encontrando. Para que no sea un paseo sin rumbo se organiza en **sesiones con carta** (*session-based testing*): un objetivo, un tiempo limitado y notas de lo hallado. Es la técnica de elección cuando la especificación es pobre o el tiempo es muy corto.

Pruebas basadas en listas de comprobación. Se sigue una lista de verificaciones derivada de la experiencia o de normas. La ventaja es la consistencia; el riesgo es el desgaste: una lista que nunca cambia deja de encontrar cosas.

7 · Enfoques colaborativos

Escribir historias de usuario en equipo. Las tres C: **Card** (la tarjeta, el enunciado), **Conversation** (la conversación que aclara), **Confirmation** (los criterios de aceptación). Y la regla INVEST: independiente, negociable, valiosa, estimable, pequeña y comprobable.

Criterios de aceptación. Definen cuándo la historia está terminada. Dos formatos comunes: orientado a escenarios (**Dado / Cuando / Entonces**) y orientado a reglas (una lista de condiciones).

Desarrollo guiado por el comportamiento (BDD). Se escriben primero los escenarios en ese formato, y sirven a la vez de especificación y de prueba.

Un matiz que cae en examen: ATDD deriva casos de prueba **desde los criterios de aceptación**, y esos casos se escriben **antes** de implementar. No es lo mismo que documentar después lo que se hizo.



V. GESTIÓN DE LA ACTIVIDAD DE PRUEBA

Planificación: qué contiene un plan de pruebas

Un plan de pruebas responde a qué se va a probar, cómo, quién, cuándo y con qué criterios se para. Lo que suele preguntarse:

- Contexto y alcance: qué entra y qué **no** entra.
- Supuestos y restricciones.
- Personas involucradas y sus responsabilidades.
- Comunicación: qué se reporta, a quién y cada cuánto.
- Análisis de riesgos y enfoque de prueba.
- Presupuesto y calendario.

Criterios de entrada y de salida

- **Criterios de entrada:** lo que tiene que existir para poder empezar. Entorno listo, código desplegado, casos escritos, datos disponibles.
- **Criterios de salida** (definición de terminado): lo que tiene que cumplirse para poder parar. Cobertura alcanzada, defectos abiertos por debajo de un umbral, todas las pruebas planificadas ejecutadas.

La frase que hay que evitar: «terminamos cuando se acabe el tiempo». Eso no es un criterio de salida, es una fecha. Un criterio de salida se puede comprobar y puede decir que **no**.

Estimación de esfuerzo

Cuatro técnicas que hay que reconocer:



- **Por ratios:** se usa un porcentaje histórico del esfuerzo de desarrollo.
- **Extrapolación:** se mide lo que costó lo hecho y se proyecta.
- **Juicio experto** (Wideband Delphi, planning poker): varias personas estiman a ciegas, comparan y convergen.
- **Tres puntos:** se calcula optimista, más probable y pesimista, y se combinan. La fórmula habitual es $(O + 4M + P) / 6$.

Ejemplo: optimista 6 días, más probable 9, pesimista 18 → $(6 + 36 + 18) / 6 = 10$ días.

Priorización: en qué orden se ejecuta

- **Basada en riesgo:** primero lo que más riesgo tiene. Es la más usada.
- **Basada en cobertura:** primero lo que más cobertura aporta.
- **Basada en requisitos:** en el orden de prioridad que fijó el negocio.

La pirámide de pruebas

Muchas pruebas rápidas y baratas abajo —unitarias—, menos en el medio —integración, servicios—, y pocas arriba —extremo a extremo por la interfaz—.

La razón no es dogma: **cuanto más arriba, más lento, más frágil y más caro de diagnosticar**. Una prueba de interfaz que falla puede tener veinte causas; una unitaria que falla tiene una.

Cuadrantes de prueba

Clasifican las pruebas en dos ejes: si **apoyan al equipo** o **critican el producto**, y si son **orientadas a negocio** o **a tecnología**.

	Orientado a tecnología	Orientado a negocio
Apoya al equipo	Q1 · unitarias, de componente. Automatizadas	Q2 · funcionales, ejemplos, historias. Automatizadas o manuales
Critica el producto	Q4 · rendimiento, seguridad, carga. Con herramientas	Q3 · exploratorias, usabilidad, aceptación. Manuales



Sirve para ver de un vistazo qué le falta a la estrategia. Un equipo con todo en Q1 y Q4 no ha hablado con el negocio.

Gestión de riesgos

Un **riesgo** es un evento posible con consecuencias negativas. Se mide por dos factores: **probabilidad** e **impacto**. El nivel de riesgo es la combinación de los dos.

Dos familias:

- **Riesgo de proyecto:** afecta a la gestión. Retrasos, falta de personal, proveedores, cambios de alcance.
- **Riesgo de producto** (riesgo de calidad): afecta al producto. Cálculos incorrectos, caídas bajo carga, fugas de datos, incumplimiento normativo.

Las **pruebas basadas en riesgo** usan ese análisis para decidir qué se prueba primero, cuánto y con qué profundidad. Es la respuesta práctica al principio de que las pruebas exhaustivas son imposibles.

Ejemplo: en una tienda en línea, el cálculo del cobro tiene impacto altísimo y probabilidad media → se prueba a fondo y temprano. El color del pie de página tiene impacto bajísimo → se mira y ya.

Monitorización, control e informes

Monitorizar es comparar lo real contra lo planeado. **Controlar** es actuar cuando se desvía.

Métricas habituales:

- Porcentaje de trabajo preparado (casos escritos, entorno listo).
- Cobertura de requisitos, de riesgos, de código.
- Defectos encontrados, corregidos, pendientes; densidad de defectos.
- Casos ejecutados, pasados, fallados, bloqueados.
- Esfuerzo y coste contra lo estimado.

Dos informes que hay que distinguir:



- **Informe de avance:** durante la ejecución. Qué se hizo, qué falta, qué bloquea, qué riesgos hay.
- **Informe de compleción:** al terminar. Resumen, evaluación contra criterios de salida, riesgos residuales, lecciones aprendidas.

Escribe los informes pensando en **quién los lee**. A la gerencia le importa el riesgo residual y la decisión; al equipo le importan los defectos y los bloqueos. Un mismo informe para los dos no sirve para ninguno.

Gestión de configuración

Suena aburrido y evita un problema muy real: **saber exactamente qué versión de qué se probó**. Todo elemento de prueba —casos, datos, entornos, herramientas— debe estar identificado, versionado y trazado. Sin eso, un defecto reportado no se puede reproducir, porque nadie sabe contra qué se probó.

Gestión de defectos

Un reporte de defecto útil lleva, como mínimo:

- Identificador y título claro.
- Fecha, autor, versión del objeto de prueba.
- **Pasos exactos para reproducirlo.**
- Resultado esperado y resultado real.
- Severidad —cuánto daña— y prioridad —cuán urgente es corregirlo—.
- Estado y referencias: requisito, caso de prueba, riesgo.

Severidad y prioridad no son lo mismo, y cae en el examen. Un error de ortografía en el logotipo de la portada es de **baja severidad** y puede ser de **prioridad altísima**. Una caída que sólo ocurre en un caso que nadie usa es de **alta severidad** y prioridad baja.



VI. HERRAMIENTAS DE PRUEBA

Para qué sirven, de verdad

Se agrupan por la actividad a la que dan soporte:

- Gestión de pruebas y de requisitos.
- Gestión de defectos.
- Gestión de configuración.
- Análisis estático.
- Diseño y generación de datos de prueba.
- Ejecución automatizada y comparación de resultados.
- Rendimiento y carga.
- Monitorización y observabilidad.
- Accesibilidad y seguridad.

Beneficios y riesgos

Beneficios	Riesgos
Reducen trabajo repetitivo	Se subestima el esfuerzo de introducirlas y mantenerlas
Dan consistencia y repetibilidad	Se sobrestima lo que la herramienta puede hacer
Miden objetivamente lo que a ojo se estima	Se confía en pruebas automatizadas mal escritas
Facilitan el acceso a la información	La herramienta pasa a mandar sobre la estrategia

La frase que hay que llevarse: una herramienta no arregla un proceso malo. Lo acelera. Automatizar un desorden produce desorden más rápido.



Automatización: qué automatizar y qué no

Buenos candidatos: regresión, pruebas de humo, casos repetitivos, cálculos con muchos datos, pruebas de carga.

Malos candidatos: lo que se ejecuta una vez, lo que cambia cada semana, lo que requiere criterio humano —usabilidad, estética, exploración—.

Y una advertencia que sale del propio temario: la automatización **no sustituye** a las pruebas manuales ni a la exploración. Cambia dónde se pone el esfuerzo humano: menos en repetir, más en pensar.

GUÍA DE ESTUDIO: TRES SEMANAS, UNA HORA AL DÍA

No hace falta más. Lo que hace falta es que sea **todos los días** y que la mitad del tiempo sea haciendo, no leyendo.

Semana 1 · Vocabulario y fundamentos

Día	Qué haces	Cómo compruebas
1	Capítulo I completo	Explica en voz alta la cadena error → defecto → fallo con un ejemplo tuyo
2	Los siete principios	Escribe una situación real de tu vida para cada uno
3	Actividades de prueba y trazabilidad	Dibuja el proceso de memoria, sin mirar
4	Capítulo II: modelos y niveles	Llena la tabla de niveles tapando la columna «quién»
5	Capítulo II: tipos, confirmación y regresión	Explica la diferencia entre confirmación y regresión sin usar la palabra «volver»
6	Capítulo III completo	Enumera los cuatro tipos de revisión y quién lidera cada uno
7	Examen 1	Corrige y anota los temas fallados

**Semana 2 · Técnicas, que es donde se gana o se pierde**

Día	Qué haces	Cómo compruebas
8	Particiones de equivalencia	Diseña particiones para tres formularios de sitios que uses a diario
9	Valores límite	Toma los mismos tres y saca los límites de dos y de tres valores
10	Tablas de decisión	Arma la tabla del préstamo con tres condiciones y colápsala
11	Transición de estados	Dibuja el diagrama de un cajero automático y saca las transiciones inválidas
12	Caja blanca: sentencias y ramas	Toma cualquier función con dos if anidados y calcula las dos coberturas
13	Basadas en experiencia y colaborativas	Escribe una carta de sesión exploratoria de 45 minutos y ejecútala
14	Examen 2	Corrige y rehaz los ejercicios fallados, no sólo leas la respuesta

**Semana 3 · Gestión, herramientas y simulacro**

Día	Qué haces	Cómo compruebas
15	Capítulo V: planificación y criterios	Escribe criterios de entrada y salida para una prueba real
16	Riesgos y priorización	Haz una matriz de riesgo con cinco riesgos de un sistema que conozcas
17	Métricas e informes	Redacta un informe de avance de media página
18	Defectos y configuración	Escribe un reporte de defecto completo de un bug real
19	Capítulo VI completo	Clasifica cinco herramientas que hayas usado
20	Repaso de lo fallado en los dos exámenes	Sólo eso. No repases lo que ya sabes
21	Examen 3, cronometrado, 60 minutos	Si sacas 32 o más, estás listo

La regla que más sube la nota: cuando falles una pregunta, no apuntes la respuesta correcta. Apunta **por qué te pareció correcta la que elegiste**. Ahí está el concepto que te falta, y es el que van a volver a preguntar.



Cinco errores que hundan a gente que sí sabía

1. **Estudiar sólo leyendo.** Los K3 se aprenden con lápiz. Si no calculaste coberturas al menos cinco veces, vas a tardar demasiado en el examen.
2. **Confundir los ejes.** Nivel, tipo y técnica son cosas distintas. «Prueba de sistema» es nivel; «no funcional» es tipo; «valores límite» es técnica.
3. **Contestar con la lógica del trabajo propio.** El examen pregunta por el temario, no por cómo se hace en tu empresa. Si en tu trabajo llaman «regresión» a otra cosa, en el examen no.
4. **Perder tiempo en las de dos y tres puntos.** Marca y sigue. Contesta primero todas las de un punto, que son la mayoría.
5. **Dejar preguntas en blanco.** No hay penalización por fallar. Una en blanco es un cero seguro; una adivinada es un 25 por ciento.

Cómo administrar los 60 minutos

- Minuto 0 a 30: las preguntas directas, sin detenerte. Marca las dudosas.
- Minuto 30 a 50: las de aplicar técnica, con calma y papel.
- Minuto 50 a 60: repasa las marcadas y **contesta todas**.

Son 90 segundos por pregunta en promedio. Suena poco y sobra tiempo si no te atorras: la mayoría se contestan en 30 segundos.



EXAMEN 1 · FUNDAMENTOS, CICLO DE VIDA Y PRUEBAS ESTÁTICAS

Veinte preguntas. Cubre los capítulos I, II y III. **Las respuestas están al final del documento**, en la hoja de respuestas 1, para que no las veas de reojo. Anota las tuyas en una hoja aparte antes de ir a comprobar.

1. ¿Cuál de las siguientes es la definición correcta de un defecto?

- a) La imperfección en un producto de trabajo que puede provocar un fallo.
- b) La equivocación que comete una persona al escribir código.
- c) La manifestación visible de un problema durante la ejecución.
- d) La razón de fondo por la que alguien se equivocó.

2. Un sistema falla en producción por una tormenta solar que altera la memoria del servidor. ¿Qué afirmación es correcta?

- a) Todo fallo tiene un defecto detrás.
- b) Es un defecto de diseño, por no prever la radiación.
- c) Un fallo puede tener causas ambientales y no un defecto.
- d) No es un fallo, porque el software estaba bien escrito.

3. El equipo ejecuta las mismas 200 pruebas automatizadas desde hace un año y hace meses que no encuentran nada. ¿Qué principio explica esto?

- a) Las pruebas exhaustivas son imposibles.
- b) Los defectos se agrupan.
- c) La ausencia de defectos es una falacia.
- d) La paradoja del pesticida.

4. ¿Cuál de estas actividades pertenece al **análisis** de pruebas?

- a) Derivar casos de prueba a partir de las condiciones.
- b) Identificar las condiciones de prueba evaluando la base de prueba.
- c) Preparar los datos de prueba y el entorno.
- d) Comparar el resultado real con el esperado.



5. ¿Qué diferencia hay entre verificación y validación?

- a) Verificar es contra la especificación; validar es contra la necesidad real.
- b) Son sinónimos dentro del temario.
- c) Verificar lo hace el equipo; validar lo hace una herramienta.
- d) Verificar es siempre estático; validar es siempre dinámico.

6. Una organización entrega un sistema sin defectos conocidos y el cliente lo rechaza porque no resuelve su problema. ¿Qué principio ilustra?

- a) Las pruebas muestran la presencia de defectos.
- b) Las pruebas tempranas ahorran tiempo y dinero.
- c) Las pruebas dependen del contexto.
- d) La ausencia de defectos es una falacia.

7. ¿Cuál de estas es una actividad de **depuración** y no de prueba?

- a) Ejecutar un caso y registrar que falló.
- b) Diseñar casos con valores límite.
- c) Localizar en el código la línea que causa el fallo y corregirla.
- d) Reportar el defecto con pasos para reproducirlo.

8. El sesgo de confirmación explica principalmente por qué:

- a) Las pruebas exhaustivas son imposibles.
- b) A quien escribió el código le cuesta ver sus propios defectos.
- c) Los defectos se agrupan en pocos módulos.
- d) Hay que renovar los casos de prueba cada cierto tiempo.

9. ¿En qué nivel de prueba se comprueba que el sistema intercambia datos correctamente con un servicio externo de facturación?



- a) Prueba de integración de sistemas.
- b) Prueba de componente.
- c) Prueba de integración de componentes.
- d) Prueba de aceptación de usuario.

10. Las pruebas **beta** se caracterizan porque:

- a) Se hacen en las instalaciones de quien desarrolla, con usuarios invitados.
- b) Las hace el equipo de desarrollo antes de liberar.
- c) Son obligatorias por contrato.
- d) Las hacen personas usuarias reales en su propio entorno.

11. Se corrigió un defecto y se ejecuta de nuevo el caso que fallaba. Eso es:

- a) Prueba de regresión.
- b) Prueba de humo.
- c) Prueba de confirmación.
- d) Prueba de aceptación operativa.

12. ¿Cuál de estas es una prueba **no funcional**?

- a) Comprobar que el descuento se calcula correctamente.
- b) Comprobar que el sistema responde en menos de dos segundos con 500 usuarios simultáneos.
- c) Comprobar que el botón «guardar» guarda.
- d) Comprobar que el flujo de compra se completa de principio a fin.

13. ¿Qué es *shift left*?

- a) Llevar las actividades de prueba lo más temprano posible en el ciclo.
- b) Mover el equipo de pruebas a otra área de la empresa.
- c) Ejecutar las pruebas de regresión antes que las funcionales.
- d) Automatizar todo lo que se pueda.

14. El análisis de impacto en pruebas de mantenimiento sirve para:



- a) Estimar el costo del proyecto completo.
- b) Decidir si el defecto es de severidad alta.
- c) Escoger la herramienta de automatización.
- d) Determinar qué partes del sistema afecta un cambio y cuánta regresión hace falta.

15. ¿Cuál de estas NO es una prueba estática?

- a) Una inspección del documento de requisitos.
- b) Ejecutar los casos de prueba de humo tras el despliegue.
- c) Un análisis estático del código con una herramienta.
- d) Una revisión técnica del diseño de arquitectura.

16. En una **inspección**, ¿quién NO debe liderar la reunión?

- a) El moderador.
- b) El líder de revisión.
- c) El autor del producto de trabajo.
- d) Un revisor experto en el dominio.

17. El beneficio principal de las pruebas estáticas frente a las dinámicas es:

- a) Encuentran defectos directamente, antes de que puedan producir un fallo.
- b) Son más rápidas de ejecutar.
- c) No necesitan personas.
- d) Sustituyen a las pruebas dinámicas.

18. Ordena correctamente las etapas de una revisión formal:

- a) Inicio → planificación → revisión individual → corrección → comunicación.
- b) Revisión individual → planificación → inicio → comunicación → corrección.
- c) Planificación → revisión individual → inicio → corrección → comunicación.
- d) Planificación → inicio → revisión individual → comunicación y análisis → corrección y reporte.

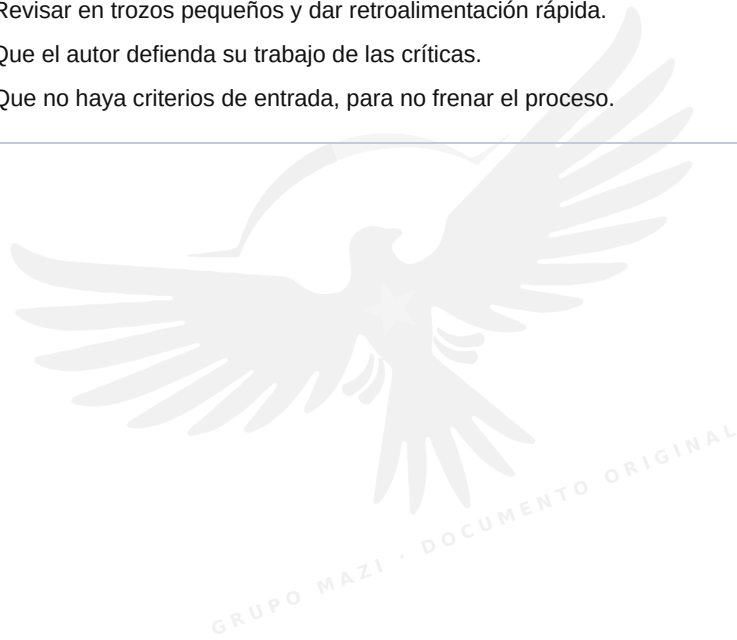
19. Una revisión **guiada** (walkthrough) se caracteriza porque:



- a) La lidera un moderador formado y produce métricas formales.
- b) No tiene proceso definido ni deja documentación.
- c) La lidera el autor y sirve para evaluar calidad, formar y consensuar.
- d) Sólo la usan los equipos de seguridad.

20. ¿Cuál de estos es un factor de éxito de las revisiones?

- a) Revisar documentos lo más grandes posible para aprovechar la reunión.
- b) Revisar en trozos pequeños y dar retroalimentación rápida.
- c) Que el autor defienda su trabajo de las críticas.
- d) Que no haya criterios de entrada, para no frenar el proceso.





EXAMEN 2 · TÉCNICAS DE DISEÑO

Veinte preguntas del capítulo IV, que es el que más pesa. Varias piden **calcular**: hazlas con papel, como en el examen real. Respuestas en la hoja de respuestas 2.

1. Un campo acepta valores de 100 a 999. ¿Cuántas particiones de equivalencia hay, contando válidas e inválidas?

- a) 2
- b) 3
- c) 4
- d) 6

2. Para el mismo campo, ¿cuáles son los valores límite con el método de **dos valores**?

- a) 100 y 999
- b) 99, 100, 101, 998, 999, 1000
- c) 99, 100, 999, 1000
- d) 0, 100, 999, 9999

3. Un sistema de envío cobra así: de 0 a 5 kg, tarifa fija; de 5.01 a 20 kg, tarifa por kilo; más de 20 kg, no se acepta. Según esa especificación, ¿cuántos casos hacen falta como mínimo para el 100 % de cobertura de particiones?

- a) 3
- b) 4
- c) 5
- d) 6

4. ¿Cuándo conviene una tabla de decisión?

- a) Cuando el sistema tiene estados que cambian con eventos.
- b) Cuando hay que medir cobertura de código.
- c) Cuando no hay especificación escrita.
- d) Cuando el resultado depende de varias condiciones combinadas.



5. Una tabla de decisión con **cuatro** condiciones booleanas tiene, sin colapsar:

- a) 4 columnas
- b) 8 columnas
- c) 16 columnas
- d) 32 columnas

6. En una tabla de decisión colapsada, un guion en una condición significa:

- a) Que la condición no aplica ni afecta al resultado en esa regla.
- b) Que la condición es siempre falsa.
- c) Que falta información.
- d) Que la regla es inválida.

7. En pruebas de transición de estados, la cobertura **0-switch** exige:

- a) Visitar cada estado al menos una vez.
- b) Ejercer cada transición válida al menos una vez.
- c) Ejercer también las transiciones inválidas.
- d) Recorrer todas las secuencias posibles de dos transiciones.

8. ¿Qué encuentra la cobertura de transiciones **inválidas** que la de válidas no encuentra?

- a) Defectos de rendimiento.
- b) Código muerto.
- c) Requisitos ambiguos.
- d) Qué hace el sistema ante un evento que no debería recibir en ese estado.

9. Observa este pseudocódigo:



Línea	Código
1	leer N
2	si $N > 0$ entonces
3	· imprimir "positivo"

Línea	Código
4	fin si
5	imprimir "fin"

Con el único caso $N = 7$, ¿qué coberturas obtienes?

- a) 100 % sentencias y 50 % ramas.
- b) 100 % sentencias y 100 % ramas.
- c) 50 % sentencias y 100 % ramas.
- d) 75 % sentencias y 50 % ramas.

10. ¿Cuál de estas afirmaciones es cierta?

- a) 100 % de cobertura de sentencias garantiza 100 % de cobertura de ramas.
- b) Las dos se garantizan mutuamente.
- c) Ninguna de las dos garantiza a la otra.
- d) 100 % de cobertura de ramas garantiza 100 % de cobertura de sentencias.

11. Este código:



Línea	Código
1	leer A, B
2	si A > 0 entonces
3	· si B > 0 entonces
4	· imprimir "ambos"

Línea	Código
5	· fin si
6	fin si

¿Cuántos casos hacen falta como mínimo para el 100 % de cobertura de ramas?

- a) 1
- b) 2
- c) 3
- d) 4

12. La técnica de **adivinación de errores** se basa en:

- a) La especificación formal del sistema.
- b) La estructura interna del código.
- c) La experiencia sobre dónde suelen aparecer defectos.
- d) Un modelo de estados.

13. Las pruebas **exploratorias** se caracterizan porque:

- a) Se diseñan por completo antes de ejecutarse.
- b) El diseño, la ejecución y la evaluación ocurren a la vez.
- c) Sólo pueden hacerse con herramientas.
- d) Sustituyen a las técnicas de caja negra.



14. ¿Qué es una **carta de sesión** en pruebas exploratorias?

- a) El objetivo y el alcance de una sesión con tiempo limitado.
- b) El reporte de defecto que se genera al final.
- c) La lista completa de casos de prueba a ejecutar.
- d) El permiso del cliente para probar en producción.

15. Las tres C de una historia de usuario son:

- a) Código, Compilación, Certificación.
- b) Cliente, Contrato, Costo.
- c) Cobertura, Criterio, Caso.
- d) Card, Conversation, Confirmation.

16. En ATDD, los casos de prueba se derivan:

- a) Del código ya escrito.
- b) De los defectos encontrados en producción.
- c) De los criterios de aceptación, **antes** de implementar.
- d) Del modelo de estados.

17. El formato **Dado / Cuando / Entonces** corresponde a criterios de aceptación:

- a) Orientados a reglas.
- b) Orientados a escenarios.
- c) Orientados a código.
- d) Orientados a riesgo.

18. Un formulario tiene un campo «tipo de cliente» con tres valores posibles y un campo «monto» con dos particiones. Si se quisiera cubrir todas las combinaciones, ¿cuántos casos serían?



- a) 5
- b) 6
- c) 9
- d) 12

19. ¿Cuál de estas técnicas es de **caja blanca**?

- a) Cobertura de ramas.
- b) Análisis de valores límite.
- c) Tabla de decisión.
- d) Transición de estados.

20. El principal valor de medir cobertura de código es:

- a) Demostrar que el software no tiene defectos.
- b) Sustituir las pruebas funcionales.
- c) Estimar el esfuerzo del proyecto.
- d) Saber qué parte del código no ejerció ninguna prueba.

GRUPO MAZI · DOCUMENTO ORIGINAL



EXAMEN 3 · SIMULACRO COMPLETO, CRONOMETRADO

Este es el que cuenta. **Cuarenta preguntas, sesenta minutos, un solo intento sin interrupciones.** Mismo reparto por capítulo que el examen oficial: ocho del I, seis del II, cuatro del III, once del IV, nueve del V y dos del VI.

Pon el cronómetro antes de leer la primera. Si te atorras más de dos minutos en una, márcala y sigue: al final vuelves. Apruebas con **26 correctas**. Respuestas y explicaciones en la hoja de respuestas 3.

Capítulo I · Fundamentos

1. El objetivo de las pruebas **cambia según el nivel y el momento**. ¿Cuál de estos objetivos es propio de las pruebas de aceptación y no de las de componente?

- a) Encontrar el mayor número de defectos posible.
- b) Verificar que cada función interna devuelve lo esperado.
- c) Generar confianza en que el producto sirve para lo que se necesita.
- d) Medir la cobertura de sentencias.

2. Alguien teclea > donde iba >=. El programa acepta pedidos de cero piezas y el almacén se queda vacío. Nombra las tres cosas, en orden:

- a) Error, defecto, fallo.
- b) Defecto, error, fallo.
- c) Fallo, defecto, causa raíz.
- d) Error, fallo, defecto.

3. ¿Cuál de estas afirmaciones sobre la **causa raíz** es correcta?

- a) Es el mismo concepto que el defecto.
- b) Es el fallo que ve la persona usuaria.
- c) Sólo aplica a defectos de seguridad.
- d) Es la razón de fondo por la que se introdujo el defecto, y corregirla evita defectos futuros.



4. Un equipo prueba un marcapasos y otro una tienda de camisetas. Los dos usan procesos muy distintos. ¿Qué principio lo explica?

- a) Los defectos se agrupan.
- b) Las pruebas dependen del contexto.
- c) La paradoja del pesticida.
- d) Las pruebas exhaustivas son imposibles.

5. Un campo de texto de 10 caracteres con 100 símbolos posibles admite 10^{20} entradas. Probarlas todas a un microsegundo cada una tardaría más que la edad del universo. ¿Qué principio se ilustra y qué se hace en su lugar?

- a) Paradoja del pesticida; se renuevan los casos.
- b) Agrupación de defectos; se prueba sólo el módulo malo.
- c) Pruebas exhaustivas imposibles; se usan técnicas y riesgo para priorizar.
- d) La ausencia de defectos es una falacia; se entrega igual.

6. ¿Cuál de estas es una actividad de **implementación** de pruebas?

- a) Identificar las condiciones de prueba.
- b) Comparar los resultados reales con los esperados.
- c) Escribir el informe de finalización.
- d) Organizar los casos en procedimientos y preparar el entorno y los datos.

7. La **trazabilidad** entre la base de prueba y los casos sirve sobre todo para:

- a) Saber el impacto de un cambio y demostrar cobertura de requisitos.
- b) Reducir el número de casos.
- c) Sustituir la documentación de diseño.
- d) Calcular la severidad de los defectos.

8. Un tester reporta un defecto y el desarrollador se lo toma como ataque personal. ¿Qué recomienda el temario?



- a) Escalar de inmediato con el gerente.
- b) Dejar de reportar defectos de ese módulo.
- c) Comunicar los hallazgos de forma neutral y centrada en el producto, no en la persona.
- d) Documentarlo por escrito y no volver a hablarlo.

Capítulo II · Pruebas en el ciclo de vida

9. En un modelo de desarrollo secuencial, ¿cuándo pueden empezar las actividades de prueba?

- a) Sólo cuando hay código ejecutable.
- b) Desde que existe la base de prueba: se puede revisar requisitos y diseñar casos.
- c) Después de la prueba de sistema.
- d) Únicamente en la fase de aceptación.

10. Entre las buenas prácticas de prueba válidas para **cualquier** ciclo de vida está:

- a) Que las pruebas se hagan sólo al final, para no repetir trabajo.
- b) Que el equipo de pruebas sea independiente de la empresa.
- c) Que se automatice el 100 % de los casos.
- d) Que a cada actividad de desarrollo le corresponda una actividad de prueba.

11. ¿Qué nivel de prueba busca defectos en las **interfaces entre módulos del mismo sistema**?

- a) Prueba de integración de componentes.
- b) Prueba de componente.
- c) Prueba de sistema.
- d) Prueba de aceptación.

12. Las **pruebas de aceptación operativa** comprueban típicamente:



- a) Que los cálculos de negocio son correctos.
- b) Que la interfaz cumple el manual de marca.
- c) Respaldo y restauración, migración de datos y tareas de mantenimiento.
- d) Que el código alcanza cierta cobertura de ramas.

13. El equipo cambia una librería de fechas que usa todo el sistema. ¿Qué tipo de prueba es imprescindible?

- a) De confirmación.
- b) De humo, únicamente.
- c) De aceptación de usuario, únicamente.
- d) De regresión.

14. ¿Cuál de estos es un **desencadenante** de pruebas de mantenimiento?

- a) La primera versión del producto.
- b) Una migración a otra plataforma o el retiro del sistema.
- c) El diseño de la arquitectura.
- d) La firma del contrato.

Capítulo III · Pruebas estáticas

15. ¿Qué productos de trabajo pueden someterse a prueba estática?

- a) Sólo el código fuente.
- b) Sólo los requisitos.
- c) Sólo lo que tenga formato ejecutable.
- d) Prácticamente cualquiera: requisitos, diseño, código, casos de prueba, contratos.

16. El **líder de revisión** y el **moderador** son roles distintos porque:



- a) El líder decide qué se revisa y organiza; el moderador conduce la reunión y media.
- b) Son sinónimos, cambia el nombre según el país.
- c) El moderador es quien escribe el producto de trabajo.
- d) El líder sólo aparece en revisiones informales.

17. ¿Cuál de estos tipos de revisión es el **más formal**?

- a) Revisión informal.
- b) Walkthrough.
- c) Revisión técnica.
- d) Inspección.

18. Un análisis estático de código puede detectar:

- a) Que el cálculo del IVA da un resultado equivocado.
- b) Que el tiempo de respuesta supera dos segundos.
- c) Variables no usadas, código inalcanzable y desviaciones del estándar de codificación.
- d) Que a la persona usuaria no le gusta el flujo.

Capítulo IV · Técnicas de prueba

19. Una promoción aplica si el cliente es socio **y** el monto supera 1000 pesos. ¿Qué técnica modela mejor esto?

- a) Tabla de decisión.
- b) Análisis de valores límite.
- c) Transición de estados.
- d) Cobertura de sentencias.

20. Un campo de edad válido acepta de 18 a 65. Con el método de **tres valores**, los valores a probar en el extremo inferior son:



- a) 18 y 19
- b) 18, 19, 20
- c) 0, 17, 18
- d) 17, 18, 19

21. ¿Cuántos casos hacen falta para el 100 % de cobertura de particiones de equivalencia en ese campo de edad, contando válidas e inválidas?

- a) 2
- b) 3
- c) 4
- d) 6

22. Un cajero automático pasa de «en espera» a «tarjeta insertada», de ahí a «PIN válido», de ahí a «operación» y regresa a «en espera». El modelo tiene 4 estados y 6 transiciones válidas. Para el 100 % de cobertura 0-switch hay que ejercer:

- a) 3 transiciones
- b) 4 transiciones
- c) 6 transiciones
- d) 12 transiciones

23. ¿Qué diferencia hay entre cobertura 0-switch y 1-switch?

- a) Ninguna, son nombres alternativos.
- b) 0-switch cubre transiciones sueltas; 1-switch cubre pares consecutivos de transiciones.
- c) 0-switch cubre estados; 1-switch cubre transiciones.
- d) 1-switch sólo aplica a transiciones inválidas.

24. Observa:

Línea	Código
1	leer edad



Línea	Código
2	si edad >= 18 entonces
3	· acceso = "permitido"
4	si no
5	· acceso = "denegado"
6	fin si
7	imprimir acceso

Contando como sentencias ejecutables las líneas 1, 2, 3, 5 y 7, con el único caso edad = 30 la cobertura de **sentencias** es:

- a) 40 %
- b) 60 %
- c) 80 %
- d) 100 %

25. Con ese mismo caso único, la cobertura de **ramas** es:

- a) 25 %
- b) 50 %
- c) 75 %
- d) 100 %

26. ¿Cuál es la relación correcta entre las dos coberturas?

- a) La de sentencias es más fuerte que la de ramas.
- b) Son independientes por completo.
- c) La de ramas sólo se mide en pruebas de sistema.
- d) La de ramas es más fuerte: alcanzar 100 % de ramas implica 100 % de sentencias.

27. ¿Cuál de estas es una técnica **basada en la experiencia**?



- a) Pruebas exploratorias.
- b) Tabla de decisión.
- c) Análisis de valores límite.
- d) Cobertura de ramas.

28. Una **lista de comprobación** (checklist) en pruebas sirve para:

- a) Sustituir la especificación.
- b) Calcular el esfuerzo del proyecto.
- c) Apoyarse en experiencia acumulada y no olvidar condiciones típicas.
- d) Medir cobertura estructural.

29. En el enfoque colaborativo, ¿quién debería escribir los criterios de aceptación?

- a) Sólo la persona de negocio.
- b) Sólo quien programa.
- c) La herramienta de gestión de pruebas.
- d) Negocio, desarrollo y pruebas juntos, en la conversación.

Capítulo V · Gestión de las pruebas

30. ¿Cuál de estos pertenece a los **criterios de salida** y no a los de entrada?

- a) Se ejecutó la cobertura planificada y los defectos abiertos están dentro del umbral acordado.
- b) El entorno de prueba está disponible.
- c) La base de prueba está aprobada.
- d) Los datos de prueba están cargados.

31. Con estimación de tres puntos, si el optimista son 6 días, el más probable 9 y el pesimista 18, la estimación es:



- a) 9 días
- b) 10 días
- c) 11 días
- d) 12 días

32. El riesgo de producto se refiere a:

- a) Que el proyecto se retrase.
- b) Que el proveedor de la herramienta quiebre.
- c) Que el producto falle y provoque daño a alguien o a la organización.
- d) Que el equipo rote.

33. En pruebas basadas en riesgo, el nivel de riesgo se calcula con:

- a) Severidad $\times$ prioridad.
- b) Número de defectos $\div$ casos ejecutados.
- c) Cobertura $\times$ esfuerzo.
- d) Probabilidad $\times$ impacto.

34. Severidad y prioridad son distintas porque:

- a) Son sinónimos con distinto nombre.
- b) La severidad es el daño que causa el fallo; la prioridad es la urgencia de corregirlo.
- c) La severidad la fija negocio; la prioridad la fija la herramienta.
- d) La prioridad sólo existe en proyectos ágiles.

35. Una errata en el logo de la portada es:

- a) Severidad baja y prioridad posiblemente alta.
- b) Severidad alta y prioridad alta.
- c) Severidad alta y prioridad baja.
- d) No es un defecto.

36. ¿Qué contiene un buen reporte de defecto?



- a) El nombre de quien lo provocó.
- b) Sólo una captura de pantalla.
- c) Identificador, pasos para reproducir, resultado esperado y real, entorno, severidad y prioridad.
- d) La corrección propuesta, escrita en código.

37. La **gestión de la configuración** en pruebas garantiza que:

- a) Los casos se ejecuten en orden alfabético.
- b) Se sepa exactamente qué versión de qué elemento se probó y se pueda reproducir.
- c) El equipo use la misma herramienta.
- d) Los defectos se cierran en 24 horas.

38. En la **pirámide de pruebas**, la base ancha corresponde a:

- a) Pruebas de interfaz de usuario, lentas y frágiles.
- b) Pruebas unitarias: muchas, rápidas y baratas.
- c) Pruebas manuales exploratorias.
- d) Pruebas de aceptación de usuario.

Capítulo VI · Herramientas

39. Un riesgo real de la automatización de pruebas es:

- a) Subestimar el mantenimiento de los scripts y confiar de más en la herramienta.
- b) Que encuentre demasiados defectos.
- c) Que las pruebas se ejecuten demasiado rápido.
- d) Que sustituya a la gestión de la configuración.

40. Antes de adoptar una herramienta en toda la organización conviene:



- a) Hacer una prueba piloto, evaluar los resultados y definir cómo se usará.
- b) Comprar la licencia más cara disponible.
- c) Automatizar primero las pruebas exploratorias.
- d) Eliminar las pruebas manuales.





HOJA DE RESPUESTAS 1 · FUNDAMENTOS, CICLO DE VIDA Y PRUEBAS ESTÁTICAS

Primero la clave rápida, para que cuentes. Debajo, el porqué de cada una: esa parte es la que de verdad te sube la calificación, porque el examen real no repite estas preguntas, repite estos razonamientos.

Pregunta	1	2	3	4	5	6	7	8	9	10
Respuesta	a	c	d	b	a	d	c	b	a	d

Pregunta	11	12	13	14	15	16	17	18	19	20
Respuesta	c	b	a	d	b	c	a	d	c	b

Tu resultado: cuenta un punto por acierto. De 14 para arriba vas bien; de 13 para abajo, relee el capítulo de las que fallaste antes de seguir.

Por qué

1 · a. El defecto vive **dentro** del producto de trabajo. La equivocación de la persona es el *error*, lo que se ve al ejecutar es el *fallo*, y la razón de fondo es la *causa raíz*. Las cuatro opciones son definiciones reales de cuatro cosas distintas: esta pregunta se cae sola si tienes la cadena clara.

2 · c. Es la excepción que el temario menciona expresamente: radiación, campos magnéticos, contaminación. Hubo un fallo, no hubo defecto. La opción a) es la trampa: suena a regla de oro y es falsa.

3 · d. Paradoja del pesticida: los mismos casos dejan de encontrar bichos porque ya mataron los que sabían matar. Se arregla revisando y ampliando los casos, no ejecutándolos más veces.

4 · b. El análisis contesta **qué** probar. Derivar los casos ya es diseño; preparar entorno y datos es implementación; comparar resultados es ejecución.

5 · a. Verificar es «¿lo construimos según la especificación?». Validar es «¿esto le sirve a quien lo va a usar?». Las dos pueden ser estáticas o dinámicas, así que d) es falsa.



- 6 · d. Sin defectos conocidos y aun así inútil. Ése es el enunciado exacto del principio de la falacia de la ausencia de defectos.
- 7 · c. Probar encuentra y reporta; depurar localiza y corrige. La frontera está en el verbo: si estás cambiando código, ya no estás probando.
- 8 · b. El sesgo de confirmación hace que uno vea lo que espera ver, y quien escribió el código espera que funcione. De ahí viene el argumento de la independencia de las pruebas.
- 9 · a. El servicio de facturación es **otro sistema**. Integración de componentes es entre módulos del mismo sistema; integración de sistemas es entre sistemas distintos. Esta distinción cae casi siempre.
- 10 · d. Alfa: en casa de quien desarrolla. Beta: en casa de quien usa. Lo que las separa es el sitio, no quién las hace.
- 11 · c. Confirmación = «¿ya quedó?». Regresión = «¿rompí algo más?». En la práctica se hacen juntas, y en el examen se preguntan separadas.
- 12 · b. Lo funcional es *qué* hace; lo no funcional es *cómo de bien* lo hace. Rendimiento con 500 usuarios es «cómo de bien».
- 13 · a. Mover las pruebas hacia la izquierda del calendario: revisar requisitos, escribir casos antes que el código. No tiene nada que ver con organigramas ni con automatizar.
- 14 · d. El análisis de impacto delimita el alcance de la regresión. Sin él sólo quedan dos opciones malas: probar todo o probar a ciegas.
- 15 · b. Estática es sin ejecutar. La prueba de humo se ejecuta, así que es dinámica. Las otras tres se hacen mirando, no corriendo.
- 16 · c. El autor nunca lidera una inspección: se lo lleva el moderador, y el autor asiste, escucha y corrige. Ojo, en el walkthrough sí lo lidera el autor — esa oposición es justo lo que se está preguntando.

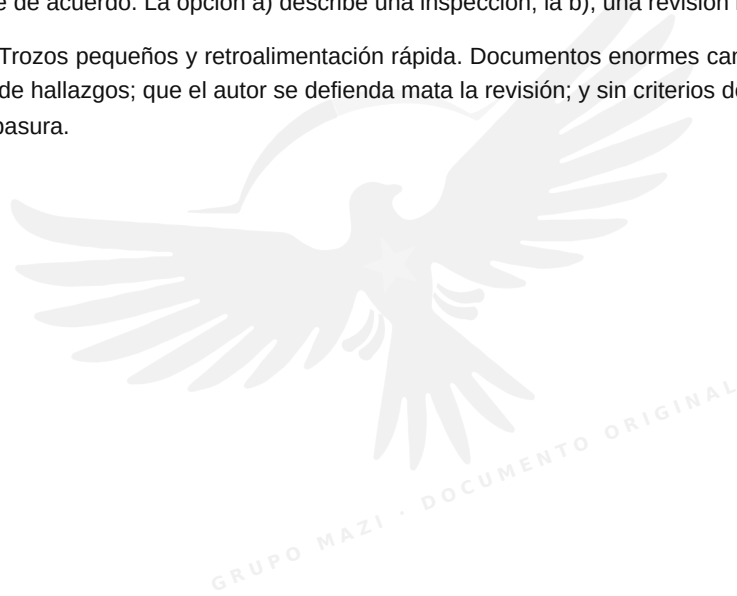


17 · a. La ventaja no es la velocidad: es que el defecto se caza **antes** de que exista siquiera la oportunidad de que provoque un fallo, y por eso sale barato. Un requisito ambiguo corregido en revisión cuesta minutos; el mismo requisito descubierto en producción cuesta una versión de emergencia.

18 · d. Planificación, inicio, revisión individual, comunicación y análisis de incidencias, corrección y reporte. Las otras tres alteran el orden en algún punto: la señal más fiable es que la revisión individual va **después** del inicio y **antes** de la reunión.

19 · c. El walkthrough lo conduce el autor y sirve tanto para evaluar como para enseñar y ponerse de acuerdo. La opción a) describe una inspección; la b), una revisión informal.

20 · b. Trozos pequeños y retroalimentación rápida. Documentos enormes cansan y bajan la tasa de hallazgos; que el autor se defienda mata la revisión; y sin criterios de entrada se revisa basura.



**HOJA DE RESPUESTAS 2 · TÉCNICAS DE DISEÑO**

Pregunta	1	2	3	4	5	6	7	8	9	10
Respuesta	b	c	a	d	c	a	b	d	a	d

Pregunta	11	12	13	14	15	16	17	18	19	20
Respuesta	c	c	b	a	d	c	b	b	a	d

Tu resultado: este examen es el más técnico. De 14 para arriba, el capítulo IV lo tienes. De 13 para abajo, vuelve a las páginas de particiones, límites y cobertura antes de intentar el simulacro completo.

Por qué

1 · b. Tres particiones: la válida (100 a 999), la inválida por abajo (menor que 100) y la inválida por arriba (mayor que 999). El error clásico es contar sólo la válida, o contar cuatro incluyendo «vacío» cuando la especificación no lo menciona.

2 · c. Dos valores: el límite y su vecino de fuera. Abajo, 100 y 99; arriba, 999 y 1000. La opción b) son seis valores, que es el método de **tres** valores; la a) se queda corta porque no cruza el límite.

3 · a. La especificación define tres particiones: 0–5, 5.01–20 y más de 20. Un caso por partición, tres casos. Si además consideraras el peso negativo como partición inválida serían cuatro, pero la especificación no lo menciona, y en el examen se cuenta lo que la especificación dice, no lo que uno imagina.

4 · d. Tabla de decisión = combinaciones de condiciones. Estados que cambian con eventos es transición de estados; cobertura de código es caja blanca.

5 · c. 2 elevado a 4 = 16 columnas. La regla es 2^n para n condiciones booleanas; con tres serían 8 y con cinco, 32. Por eso se colapsan: 16 columnas ya son incómodas y con seis condiciones tendrías 64.

6 · a. El guion es «da igual»: en esa regla, el resultado no depende de esa condición. Es exactamente lo que permite colapsar la tabla.



7 · b. 0-switch = cada transición **válida** una vez. Visitar cada estado es menos exigente; las secuencias de dos transiciones son 1-switch; las inválidas son una cobertura aparte.

8 · d. Las transiciones inválidas contestan la pregunta interesante: ¿qué pasa si mando un evento que no toca? Ahí es donde salen los sistemas que se quedan colgados o hacen algo absurdo.

9 · a. Sentencias ejecutables: leer N, la decisión, imprimir "positivo" e imprimir "fin". Con $N = 7$ se ejecutan las cuatro, o sea 100 %. Ramas: la decisión tiene dos salidas y sólo recorriste la verdadera, o sea 50 %. Esta pregunta, con otros números, aparece en casi todos los simulacros.

10 · d. Ramas es la cobertura más fuerte de las dos: si recorriste todas las salidas de todas las decisiones, forzosamente pisaste todas las sentencias. Al revés no: el ejemplo de la pregunta 9 lo demuestra con 100 % de sentencias y 50 % de ramas.

11 · c. Tres casos: ($A > 0$, $B > 0$) cubre las dos salidas verdaderas; ($A > 0$, $B \leq 0$) cubre la falsa de la interna; ($A \leq 0$) cubre la falsa de la externa. Con dos casos siempre te queda una salida sin recorrer, porque para llegar a la decisión interna hace falta que la externa sea verdadera.

12 · c. Adivinación de errores = experiencia. «Aquí siempre meten el campo vacío», «esta fecha va a reventar en febrero». Se apoya en listas de defectos típicos, no en la especificación ni en el código.

13 · b. Exploratorias: se diseña, se ejecuta y se evalúa a la vez, dentro de una sesión con tiempo limitado. No son «probar sin pensar»: llevan carta de sesión y bitácora.

14 · a. La carta de sesión dice qué se va a explorar, con qué objetivo y por cuánto tiempo. Es lo que separa la prueba exploratoria del picoteo.

15 · d. Card (la tarjeta), Conversation (la conversación) y Confirmation (los criterios de aceptación). La tarjeta es la excusa para la conversación, y la confirmación es lo que la convierte en algo probable.

16 · c. En ATDD los criterios de aceptación se convierten en casos **antes** de escribir el código, y por eso el código nace ya con su prueba.

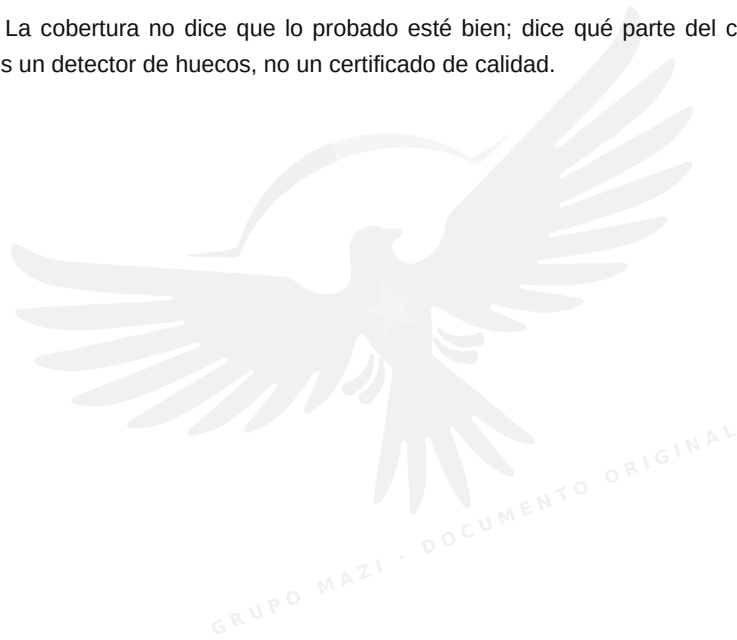


17 · b. Dado / Cuando / Entonces describe un escenario concreto. Los criterios orientados a reglas son listas de condiciones, sin narrativa.

18 · b. $3 \times 2 = 6$ combinaciones. La trampa es sumar (5) o elevar al cuadrado (9). Y ojo con el enunciado: pide **todas** las combinaciones, no la cobertura mínima de particiones, que serían $3 + 2 = 5$ casos bien repartidos.

19 · a. Cobertura de ramas mira la estructura interna, así que es caja blanca. Las otras tres se diseñan desde la especificación, sin abrir el código.

20 · d. La cobertura no dice que lo probado esté bien; dice qué parte del código **nadie tocó**. Es un detector de huecos, no un certificado de calidad.



**HOJA DE RESPUESTAS 3 · SIMULACRO COMPLETO**

Pregunta	1	2	3	4	5	6	7	8	9	10
Respuesta	c	a	d	b	c	d	a	c	b	d

Pregunta	11	12	13	14	15	16	17	18	19	20
Respuesta	a	c	d	b	d	a	d	c	a	d

Pregunta	21	22	23	24	25	26	27	28	29	30
Respuesta	b	c	b	c	b	d	a	c	d	a

Pregunta	31	32	33	34	35	36	37	38	39	40
Respuesta	b	c	d	b	a	c	b	b	a	a

Tu resultado, con la misma vara que el examen oficial:

Aciertos	Qué significa
26 o más	Aprobado. Preséntalo.
22 a 25	Cerca. Repasa los capítulos de las que fallaste y repite el simulacro en una semana.
21 o menos	Todavía no. Vuelve a la guía completa antes de volver a examinarte.

Por qué

1 · c. Cada nivel tiene su objetivo. En componente se busca defectos y cobertura; en aceptación se busca **confianza** y evidencia de que el producto sirve. Las opciones a) y b) son objetivos de niveles tempranos y la d) es una métrica de caja blanca.

2 · a. La cadena siempre es la misma: alguien se equivoca (**error**), eso deja algo mal en el producto (**defecto**), y al ejecutar se manifiesta (**fallo**). Si te la aprendes en ese orden, tres o cuatro preguntas del examen real se contestan solas.



- 3 • d. La causa raíz explica **por qué** apareció el defecto: una especificación ambigua, falta de formación, prisa. Corregir el defecto arregla un caso; corregir la causa raíz evita los siguientes.
- 4 • b. Marcapasos y camisetas no se prueban igual porque el riesgo no es el mismo. Ése es el principio de dependencia del contexto, y es el que justifica que no haya una receta única.
- 5 • c. Es el argumento clásico de la imposibilidad de las pruebas exhaustivas: el espacio de entradas es astronómico. Por eso existen las técnicas de diseño y la priorización por riesgo.
- 6 • d. Implementación es la carpintería: armar procedimientos, cargar datos, dejar el entorno listo. Identificar condiciones es análisis, comparar resultados es ejecución, y el informe es cierre.
- 7 • a. La trazabilidad sirve para dos cosas concretas: analizar impacto cuando algo cambia, y demostrar ante quien pregunte qué requisito está cubierto por qué caso.
- 8 • c. El temario es explícito con esto: hablar del producto, no de la persona, y presentar el hallazgo como información, no como acusación. Es contenido de examen, no cortesía.
- 9 • b. En cascada, las pruebas **dinámicas** esperan al código, pero las actividades de prueba empiezan con el primer documento revisable. Confundir «prueba» con «ejecutar» es lo que hace fallar esta pregunta.
- 10 • d. La correspondencia una a una entre actividad de desarrollo y actividad de prueba es la buena práctica que aplica a cualquier modelo. Las demás opciones son o malas prácticas o exageraciones.
- 11 • a. Entre módulos del mismo sistema: integración de componentes. Entre sistemas distintos: integración de sistemas. Es la misma distinción de la pregunta 9 del examen 1, planteada al revés.
- 12 • c. La aceptación operativa la hace quien va a operar el sistema: respaldos, restauración, migraciones, tareas programadas, seguridad de operación. No mira reglas de negocio.



- 13 · d.** Cambiar algo que usa todo el sistema es el caso de libro de la regresión: no estás comprobando la corrección de un defecto, estás comprobando que no rompiste lo que ya funcionaba.
- 14 · b.** Los desencadenantes de mantenimiento son modificaciones, migraciones y retiro. La primera versión no es mantenimiento: es desarrollo.
- 15 · d.** Todo producto de trabajo legible es candidato: requisitos, historias de usuario, diseño, código, casos de prueba, guías, contratos, hasta modelos. Es una de las preguntas más fáciles del capítulo III y se falla por leer de prisa.
- 16 · a.** El líder de revisión decide el alcance, las personas y el momento; el moderador conduce la sesión, cuida el tono y media si hay fricción. En equipos pequeños los junta la misma persona, pero el temario los distingue.
- 17 · d.** La escala va de menos a más formal: informal, walkthrough, técnica, inspección. La inspección es la única con roles definidos, métricas y criterios de entrada y salida.
- 18 · c.** Una herramienta de análisis estático ve la estructura del código: variables sin usar, código inalcanzable, complejidad, incumplimiento del estándar. Lo que **no** puede ver es si la lógica de negocio es correcta.
- 19 · a.** Dos condiciones combinadas con «y» piden tabla de decisión. Cuatro combinaciones posibles, cuatro columnas.
- 20 · d.** Tres valores: el límite y sus dos vecinos. Para el 18 son 17, 18 y
- 1.** La opción a) es el método de dos valores.
- 21 · b.** Tres particiones: menor que 18, de 18 a 65, mayor que 65. Un caso por partición.
- 22 · c.** 0-switch es una prueba por transición válida. Hay 6, hacen falta 6. La opción b) es el número de **estados**, que es la confusión que la pregunta está buscando.
- 23 · b.** 0-switch: cada transición suelta. 1-switch: cada par de transiciones consecutivas, que es donde salen los defectos de secuencia. El número del nombre es cuántos «saltos extra» encadenas.
- 24 · c.** Con edad = 30 se ejecutan las líneas 1, 2, 3 y 7. La 5 no. Cuatro de cinco sentencias = 80 %.



- 25 · **b.** La decisión tiene dos salidas y sólo recorriste la verdadera: 50 %. Para llegar al 100 % necesitas un segundo caso, por ejemplo edad = 15.
- 26 · **d.** Ramas es la más fuerte. Las preguntas 24 y 25 son la demostración: 80 % de sentencias con 50 % de ramas. Nunca verás lo contrario.
- 27 · **a.** Exploratorias, adivinación de errores y listas de comprobación son las tres basadas en experiencia. Las otras opciones son de especificación o de estructura.
- 28 · **c.** La checklist es memoria de equipo: condiciones que ya nos mordieron alguna vez y no queremos volver a olvidar.
- 29 · **d.** Los criterios de aceptación salen de la conversación entre las tres partes. Si los escribe una sola, vuelves a tener el problema que el enfoque colaborativo quiere evitar.
- 30 · **a.** Entrada es lo que hace falta **para empezar**; salida es lo que hace falta **para dar por terminado**. Cobertura ejecutada y defectos bajo umbral es salida; entorno, base de prueba y datos son entrada.
- 31 · **b.** $(6 + 4 \times 9 + 18) \div 6 = (6 + 36 + 18) \div 6 = 60 \div 6 = 10$ días. El error frecuente es promediar los tres a secas, que daría 11.
- 32 · **c.** Riesgo de producto = el producto falla y hace daño. Riesgo de proyecto = el proyecto se retrasa, se queda sin gente o sin presupuesto. Las opciones a), b) y d) son las tres de proyecto.
- 33 · **d.** Probabilidad de que ocurra por impacto si ocurre. Severidad y prioridad son atributos del defecto ya encontrado, que es otra cosa.
- 34 · **b.** Severidad: cuánto daño hace el fallo. Prioridad: con cuánta urgencia hay que arreglarlo. La decide el negocio y puede no coincidir con la severidad, como demuestra la pregunta siguiente.
- 35 · **a.** El sistema funciona perfecto, así que la severidad es baja. Pero sale en la portada y lo ve todo el mundo, así que puede ser lo primero que se corrija. Es el ejemplo canónico de severidad baja con prioridad alta.
- 36 · **c.** El reporte tiene que permitir **reproducir** y **decidir**: pasos, esperado, real, entorno, severidad y prioridad. Nunca lleva culpables ni promete la corrección.



37 · b. Sin gestión de configuración no sabes qué versión probaste, y un resultado que no se puede reproducir no sirve como evidencia.

38 · b. Base ancha: muchas pruebas unitarias, rápidas y baratas. Punta estrecha: pocas de interfaz, lentas y frágiles. La pirámide invertida es justamente el antipatrón.

39 · a. El costo escondido de automatizar es el mantenimiento: la suite envejece con el producto. El segundo riesgo es confiar de más y dejar de mirar.

40 · a. Piloto primero: en un proyecto acotado, con criterios de evaluación, y luego se decide. Comprar para toda la organización sin piloto es la forma más cara de descubrir que la herramienta no encajaba.





| GLOSARIO · LOS TÉRMINOS QUE SÍ CAEN

No están todos los del glosario oficial, que pasa de mil entradas. Están los que aparecen en las preguntas, con la definición en el idioma en que se piensa, no en el que se traduce. Al lado, entre paréntesis, el término en inglés, porque el examen se puede presentar en inglés y porque los materiales que vas a encontrar después están casi todos en ese idioma.

Los cuatro que se confunden entre sí

- **Error** (*error, mistake*) — la equivocación humana. Alguien entendió mal, se distrajo o tecleó de más.
- **Defecto** (*defect, fault, bug*) — la imperfección que queda dentro del producto de trabajo. Puede llevar años ahí sin que nadie lo note.
- **Fallo** (*failure*) — lo que se ve cuando el defecto se ejecuta. Es el único de los cuatro que ocurre en tiempo real.
- **Causa raíz** (*root cause*) — por qué se introdujo el defecto. Corregirla evita la familia entera de defectos parecidos.



Fundamentos

- **Base de prueba** (*test basis*) — todo lo que sirve para saber qué debería hacer el sistema: requisitos, historias, diseño, contratos, incluso el sistema anterior.
- **Condición de prueba** (*test condition*) — un aspecto concreto que hay que probar. Todavía no es un caso: es «hay que probar el descuento de socio».
- **Caso de prueba** (*test case*) — condiciones previas, entradas, resultado esperado y condiciones posteriores. Si no tiene resultado esperado, no es un caso de prueba.
- **Oráculo de prueba** (*test oracle*) — la fuente que dice cuál es el resultado correcto. Puede ser la especificación, el sistema anterior o una persona experta.
- **Cobertura** (*coverage*) — qué porcentaje de un conjunto definido (sentencias, ramas, requisitos, particiones) ejercieron las pruebas.
- **Producto de trabajo de prueba** (*test work product*) — cualquier cosa que produce el proceso de pruebas: plan, casos, datos, reportes, informes.
- **Verificación** (*verification*) — ¿lo construimos conforme a lo especificado?
- **Validación** (*validation*) — ¿esto resuelve lo que la persona necesitaba?
- **Trazabilidad** (*traceability*) — el hilo que conecta requisito, condición, caso, ejecución y defecto. Sirve para análisis de impacto y para demostrar cobertura.
- **Depuración** (*debugging*) — localizar y corregir el defecto. No es una actividad de prueba.



Ciclo de vida y niveles

- **Shift left** — llevar las actividades de prueba lo más temprano posible.
- **Prueba de componente** (*component / unit testing*) — un módulo aislado.
- **Integración de componentes** (*component integration testing*) — las interfaces entre módulos del mismo sistema.
- **Integración de sistemas** (*system integration testing*) — las interfaces con otros sistemas o servicios externos.
- **Prueba de sistema** (*system testing*) — el comportamiento del sistema completo.
- **Aceptación de usuario** (*user acceptance testing, UAT*) — la hace quien va a usarlo, para decidir si lo acepta.
- **Aceptación operativa** (*operational acceptance testing*) — respaldos, restauración, migración, tareas de mantenimiento.
- **Prueba alfa** (*alpha testing*) — en las instalaciones de quien desarrolla.
- **Prueba beta** (*beta testing*) — en el entorno real de quien usa.
- **Prueba de confirmación** (*confirmation / re-testing*) — comprobar que la corrección funcionó.
- **Prueba de regresión** (*regression testing*) — comprobar que el cambio no rompió lo que ya funcionaba.
- **Prueba de humo** (*smoke testing*) — un puñado de casos rápidos para saber si vale la pena seguir probando esta versión.
- **Prueba funcional** — qué hace el sistema.
- **Prueba no funcional** — cómo de bien lo hace: rendimiento, usabilidad, seguridad, portabilidad, fiabilidad, mantenibilidad, compatibilidad.
- **Caja negra** (*black-box*) — se diseña desde la especificación, sin mirar el código.
- **Caja blanca** (*white-box*) — se diseña desde la estructura interna.
- **Análisis de impacto** (*impact analysis*) — determinar qué toca un cambio, para dimensionar la regresión.



Pruebas estáticas

- **Prueba estática** (*static testing*) — examinar sin ejecutar.
- **Revisión informal** — sin proceso ni documentación obligatoria.
- **Walkthrough / revisión guiada** — la conduce el autor; sirve para evaluar, formar y consensuar.
- **Revisión técnica** — la conducen personas técnicas, con documentación y decisiones registradas.
- **Inspección** — la más formal: roles definidos, moderador, métricas, criterios de entrada y salida.
- **Análisis estático** (*static analysis*) — una herramienta revisa el código sin ejecutarlo: variables sin usar, código inalcanzable, complejidad, desviaciones del estándar.
- **Moderador** (*moderator, facilitator*) — conduce la reunión y media.
- **Escriba** (*scribe*) — registra las incidencias encontradas.

Técnicas de caja negra

- **Partición de equivalencia** (*equivalence partitioning*) — agrupar entradas que el sistema trata igual, y probar una de cada grupo.
- **Análisis de valores límite** (*boundary value analysis, BVA*) — probar los bordes de cada partición, que es donde se rompe. Método de **dos valores**: el límite y su vecino de fuera. Método de **tres valores**: el límite y sus dos vecinos.
- **Tabla de decisión** (*decision table*) — filas de condiciones y acciones, columnas de reglas. Sin colapsar tiene 2^n columnas para n condiciones booleanas.
- **Transición de estados** (*state transition testing*) — estados, eventos, transiciones y acciones. Cobertura **0-switch**: cada transición válida. Cobertura **1-switch**: cada par de transiciones consecutivas.

Técnicas de caja blanca



- **Cobertura de sentencias** (*statement coverage*) — porcentaje de sentencias ejecutables que ejercieron las pruebas.
- **Cobertura de ramas** (*branch coverage*) — porcentaje de salidas de decisión ejercidas. Es más fuerte que la de sentencias.

Técnicas basadas en la experiencia

- **Adivinación de errores** (*error guessing*) — usar la experiencia sobre dónde suelen esconderse los defectos.
- **Prueba exploratoria** (*exploratory testing*) — diseñar, ejecutar y evaluar a la vez, dentro de una sesión acotada.
- **Carta de sesión** (*test charter*) — el objetivo, el alcance y el tiempo de una sesión exploratoria.
- **Prueba basada en listas de comprobación** (*checklist-based testing*) — cubrir una lista de condiciones acumulada por experiencia.

Enfoque colaborativo

- **Las tres C** — Card, Conversation, Confirmation: la tarjeta de la historia, la conversación que la aclara y los criterios que la hacen comprobable.
- **INVEST** — Independiente, Negociable, Valiosa, Estimable, Small (pequeña), Testeable. Las seis propiedades de una buena historia de usuario.
- **ATDD** (*acceptance test-driven development*) — los casos se derivan de los criterios de aceptación antes de escribir el código.
- **Dado / Cuando / Entonces** (*Given / When / Then*) — formato de criterio de aceptación orientado a escenarios.



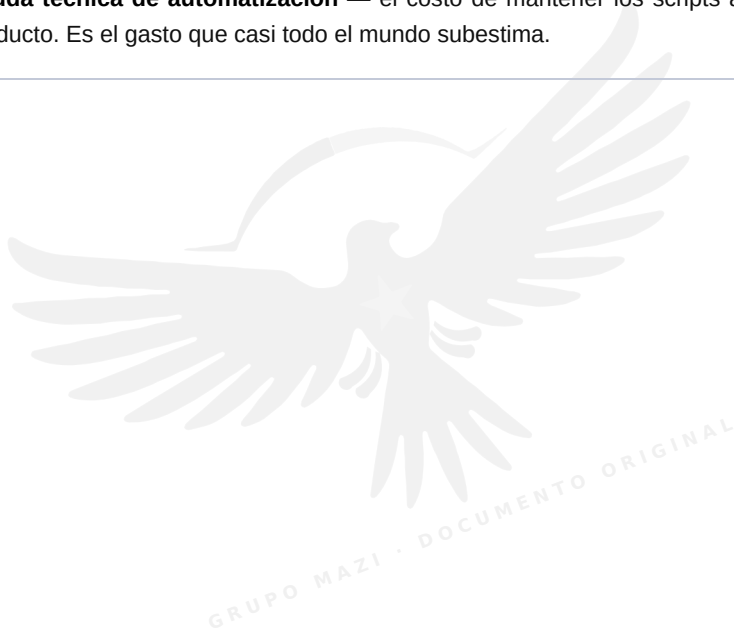
Gestión

- **Plan de pruebas** (*test plan*) — alcance, objetivos, riesgos, enfoque, recursos, calendario, criterios de entrada y salida.
- **Criterios de entrada** (*entry criteria*) — lo que hace falta para empezar.
- **Criterios de salida** (*exit criteria*, también *definition of done*) — lo que hace falta para dar por terminado.
- **Estimación de tres puntos** — $(\text{optimista} + 4 \times \text{probable} + \text{pesimista}) \div 6$.
- **Riesgo de proyecto** — amenaza a la ejecución del proyecto: calendario, presupuesto, gente.
- **Riesgo de producto** — amenaza a la calidad del producto y al daño que puede causar.
- **Nivel de riesgo** — probabilidad $\times$ impacto.
- **Prueba basada en riesgo** (*risk-based testing*) — se prueba más y antes lo que más riesgo tiene.
- **Pirámide de pruebas** (*test pyramid*) — muchas unitarias en la base, pocas de interfaz en la punta.
- **Cuadrantes de prueba** (*testing quadrants*) — cuatro grupos según si la prueba apoya al equipo o critica el producto, y si es de negocio o técnica.
- **Severidad** (*severity*) — cuánto daño causa el fallo.
- **Prioridad** (*priority*) — con cuánta urgencia hay que corregirlo. La fija el negocio y puede no coincidir con la severidad.
- **Gestión de la configuración** (*configuration management*) — saber qué versión de qué elemento se probó, para poder reproducirlo.
- **Informe de progreso** (*test progress report*) — el estado durante la ejecución.
- **Informe de finalización** (*test completion report*) — el resumen al cerrar.



Herramientas

- **Herramienta de ejecución** — ejecuta casos automatizados y compara resultados.
- **Arnés de prueba** (*test harness*) — el andamiaje que permite ejecutar un componente aislado, con *stubs* y *drivers*.
- **Integración continua** (*continuous integration, CI*) — cada cambio se integra y se prueba automáticamente.
- **Deuda técnica de automatización** — el costo de mantener los scripts al día con el producto. Es el gasto que casi todo el mundo subestima.





¿DÓNDE PRESENTAR EL EXAMEN Y QUIÉN DA EL CURSO

Esta lista no la inventé ni la saqué de un buscador. Está tomada del **padrón oficial de proveedores de formación acreditados de HASTQB** —el consejo ISTQB para Hispanoamérica— consultado el 30 de agosto de 2026, entrando ficha por ficha para sacar el teléfono y el correo de cada uno.

Dos advertencias honestas antes de que marques:

- **Los datos envejecen.** Un teléfono correcto hoy puede no serlo en seis meses. Si uno no contesta, el siguiente de la lista da el mismo curso.
- **Donde no aparece el país es porque la ficha de origen no lo declaraba.** No lo deduje del prefijo telefónico, porque deducirlo es inventarlo. El prefijo está ahí y lo puedes leer tú.

Son **23 centros acreditados que imparten el Foundation Level (CTFL)**. Pide siempre dos cosas al llamar: si el precio incluye el **voucher del examen**, y si el curso es en **español**.

CENTROS ACREDITADOS CON CTFL

Verity Consulting

País	Chile
Teléfono	+56 2 27148720
Correo	karina.meza@verity.cl
Sitio	http://www.verity.cl/

ATESOFT SAC

Teléfono	51 4057909
Correo	informes@atesoftperu.com
Sitio	www.atesoftperu.com



Babel Learning Center S.A.

Teléfono 4001 2834
Correo learning@grupobabel.com
Sitio www.babel-learning.com

Calidad de conceptos SAC

Teléfono: +51 978904540 51 980917045

Correo info@cdconceptos.com
Sitio www.cdconceptos.com

Choucair Testing

Teléfono +574 4480510
Correo cursoschoucair@choucairtesting.com
Sitio www.choucairtesting.com

Cool Testers (Aurean Blue)

País México
Teléfono +52 8130788589
Correo contacto@cooltesters.com
Sitio <http://www.cooltesters.com/>



Creativa Consultores

País El Salvador
Teléfono (503) 2217-3800
Correo hi@creativastudios.us
Sitio www.creativaconsultores.com

ECOSISTEMAS

País Argentina
Teléfono +54-1126809149
Correo agrano@ecosistemas.com.ar
Sitio https://ecosistemasglobal.com/

GESTIÓN IT S.R.L

Teléfono (54 11) 5031-8810
Correo ruben.duro@gestionit.com.ar
Sitio www.gestionit.com.ar

GLOBAL RESOURCES

Teléfono +58 212 615-3645
Correo contacto@globalr.net
Sitio www.globalr.net

IT ALIANZA S.A.



Teléfono (595984)175112
Correo info@italianza.com.py
Sitio www.italianza.com.py

JB ENTERPRISE GROUP S.A.C.

Teléfono +51 951 208 836
Correo cursos@jbenterprisegroup.com
Sitio www.jbenterprisegroup.com

LASERANTS

Teléfono (502) 3082 9898
Correo info@laserants.com
Sitio www.laserants.com

Logic Studio S.A.

País Panamá
Teléfono +507 317 1258
Correo yaitzy.julio@logicstudio.net
Sitio www.logicstudio.net

MTP Digital Business Assurance

País México
Teléfono +52 1 55 5286 1461
Correo campus@mtpinternational.mx
Sitio https://mtpinternational.mx/



QA Minds Lab

País México
Teléfono +52 33 1820 0455 – 3411591743
Correo gbejines@qamindslab.com

QArmy Academia de QA

País Argentina
Teléfono +54 9 261 5658888
Correo info@qarmy.ar

Quality Data S.A.

Teléfono +57 1 6377060
Sitio www.qdata.com.co

Servicios Informáticos y Capacitación SPA

Teléfono 96676690
Correo cursos@testingenchile.cl
Sitio <https://www.testingenchile.cl/>

SJ Agile Consulting

Teléfono +51961356046 → Sandra Villanueva
Correo cursos@sjagileconsulting.com



TECNOLOGÍA APLICADA, S.A. (TECNASA)

Teléfono (+507) 366-6888
Correo tecnasau@tecnasa.com
Sitio <https://tecnasau.tecnasa.com/>

TestGroup S.A.

País Chile
Teléfono +56 9 6326 2990
Correo romina.campos@testgroup.cl
Sitio www.testgroup.cl

Testing IT Consulting S.A. de C.V.

Teléfono +52 5566-3535
Correo info@testingit.com.mx
Sitio www.testingit.com.mx

GRUPO MAZI · DOCUMENTO ORIGINAL



OTROS CENTROS DEL MISMO PADRÓN

Estos aparecen en el padrón de HASTQB como proveedores acreditados, pero su ficha **no listaba CTFL** el día de la consulta. Vale la pena preguntarles: los catálogos cambian antes que los directorios.

Business Innovations S.R.L.

Teléfono +511 2260325
Correo info@businessinnova.com
Sitio www.businessinnova.com

CENFOTEC IT LEARNING CENTER

Teléfono + 506 4000-3950
Correo rchi@ucenfotec.ac.cr
Sitio www.ucenfotec.ac.cr

FYC Soluciones Integrales, C.A. (Grupo FYC)

Teléfono +58 212 232-7811
Correo informacion@fyccorp.com
Sitio www.fyccorp.com

Hypernova Labs S.A.

Correo info@hypernovallabs.com
Sitio <https://www.hypernovallabs.com/>

**ITWorkers SA DE CV**

País México
Teléfono +52 72 9556 5103
Correo juan.vargas@itw.mx
Sitio www.itw.mx

QA Team

Teléfono +51 992 062 484
Correo info@gateamperu.com
Sitio https://gateamperu.com/

Turing Ti

País Chile
Teléfono (+56) 9 54076772
Correo daniel.tolosa@turingti.com
Sitio www.turingti.com

LOS CONSEJOS, QUE ES A DONDE SE LLAMA CUANDO NADIE CONTESTA

HASTQB · Hispanic America Software Testing Qualifications Board

Es el consejo que acredita a todos los centros de arriba y el que publica el padrón del que salió esta lista. Si un proveedor no contesta o quieres confirmar que sigue acreditado, se pregunta aquí.

Sitio <https://hastqb.org/>

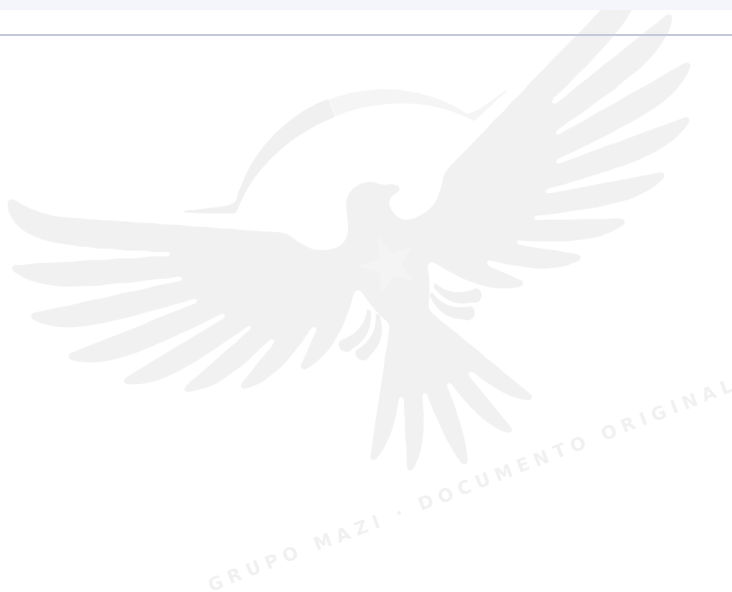


ISTQB · International Software Testing Qualifications Board

El organismo global. Ahí está el temario oficial, el glosario y el buscador de consejos por país, por si el empleado de Grupo Mazi se presenta fuera de Hispanoamérica.

Sitio <https://www.istqb.org/>

Un consejo que no viene en ningún directorio: llama a tres, no a uno. El precio del mismo examen varía bastante entre centros, y algunos regalan el segundo intento si repruebas. Eso no lo publican: se pregunta.





PARA LEER DESPUÉS · ENLACES COMPROBADOS

Con esta guía tienes de sobra para presentar el examen: no hace falta abrir ninguna de estas páginas para aprobar. Están aquí para lo que viene **después** del certificado, que es donde de verdad se aprende el oficio.

Cada dirección de esta lista se abrió y se comprobó el 30 de agosto de 2026. No hay ninguna puesta de memoria. Si alguna deja de funcionar más adelante, es que el sitio la movió, no que estuviera mal escrita.

Lo oficial, que manda sobre todo lo demás

- **Temario oficial del Foundation Level v4.0** — la fuente de la que sale todo lo de esta guía. Es un PDF gratuito y está en varios idiomas. <https://www.istqb.org/certifications/certified-tester-foundation-level-ctfl-v4-0>
- **Página de la certificación CTFL** — requisitos, formato del examen y por dónde se inscribe uno. <https://www.istqb.org/certifications/certified-tester-foundation-level>
- **Glosario oficial de ISTQB** — la definición exacta de cada término, que es la que se usa para redactar las preguntas. Cuando dudes de una palabra, aquí se acaba la discusión. <https://glossary.istqb.org/>
- **Catálogo completo de certificaciones ISTQB** — lo que viene después del Foundation: los niveles Advanced y las especialidades. <https://www.istqb.org/certifications/>
- **HASTQB** — el consejo para Hispanoamérica y el padrón de centros acreditados del que salió el directorio de la sección anterior. <https://hastqb.org/>

Capítulo IV · las técnicas, explicadas por fuera del temario

Leerlas contadas de otra manera es la forma más rápida de que se queden.



- **Partición de equivalencia** — con ejemplos distintos a los míos. https://en.wikipedia.org/wiki/Equivalence_partitioning
- **Análisis de valores límite** — de dónde sale la idea de que los defectos viven en los bordes. https://en.wikipedia.org/wiki/Boundary-value_analysis
- **Tablas de decisión** — el formato completo, incluido el colapsado. https://en.wikipedia.org/wiki/Decision_table
- **Tablas de transición de estados** — la representación tabular del mismo modelo que dibujaste con círculos y flechas. https://en.wikipedia.org/wiki/State_transition_table
- **Cobertura de código** — los tipos de cobertura, incluidos los que el Foundation no pide y el Advanced sí. https://en.wikipedia.org/wiki/Code_coverage
- **Pruebas de pares (all-pairs)** — no entra en el examen, pero es la técnica que usarías de verdad cuando las combinaciones se disparan. https://en.wikipedia.org/wiki/All-pairs_testing
- **Pruebas exploratorias** — la historia y la crítica de la técnica. https://en.wikipedia.org/wiki/Exploratory_testing

Capítulos II y III · niveles, tipos y revisiones

- **Pruebas de regresión** — por qué existen y por qué se automatizan primero. https://en.wikipedia.org/wiki/Regression_testing
- **Pruebas de aceptación** — la diferencia entre aceptación de usuario, contractual, regulatoria, alfa y beta. https://en.wikipedia.org/wiki/Acceptance_testing
- **Shift-left** — de dónde viene el término y qué significa fuera del examen. https://en.wikipedia.org/wiki/Shift-left_testing
- **Inspección de software** — el método formal de Fagan, que es el abuelo de la inspección que estudiaste. https://en.wikipedia.org/wiki/Software_inspection
- **Análisis estático** — qué puede y qué no puede ver una herramienta que lee código sin ejecutarlo. https://en.wikipedia.org/wiki/Static_program_analysis

Capítulo V · la pirámide y la gestión, por quienes la inventaron



Martin Fowler es la referencia canónica de casi todo lo que el temario da por sabido en este capítulo. Está en inglés y vale cada minuto.

- **The Practical Test Pyramid** — el artículo largo, con código. Si sólo vas a leer uno de esta lista, que sea éste. <https://martinfowler.com/articles/practical-test-pyramid.html>
- **Test Pyramid** — la versión corta, de dos minutos, para tener la idea. <https://martinfowler.com/bliki/TestPyramid.html>
- **On the Diverse and Fantastical Shapes of Testing** — la crítica honesta a la pirámide: cuándo no aplica y qué formas alternativas funcionan. <https://martinfowler.com/articles/2021-test-shapes.html>
- **Unit Test** — por qué «prueba unitaria» significa cosas distintas según con quién hables. Sirve para no discutir de balde en el trabajo. <https://martinfowler.com/bliki/UnitTest.html>
- **Test Double** — dobles, mocks, stubs y fakes, cada uno con su nombre correcto. <https://martinfowler.com/bliki/TestDouble.html>
- **Self Testing Code** — la idea de que el código traiga sus propias pruebas. <https://martinfowler.com/bliki/SelfTestingCode.html>
- **Continuous Integration** — el artículo original que definió la práctica que el capítulo VI da por hecha. <https://martinfowler.com/articles/continuousIntegration.html>
- **Estimación de tres puntos** — la fórmula que calculaste en el examen 3, con su origen en PERT. https://en.wikipedia.org/wiki/Three-point_estimation

Enfoque colaborativo · ATDD y BDD en su casa

- **Referencia de Gherkin** — la sintaxis Dado / Cuando / Entonces, en el proyecto que la popularizó. Trae la traducción al español. <https://cucumber.io/docs/gherkin/reference/>
- **Introducción a BDD** — qué es y en qué se diferencia de escribir pruebas. <https://cucumber.io/docs/bdd/>



Para seguir después del certificado

- **Software testing, panorama general** — el mapa completo de la disciplina, con las ramas que el Foundation sólo menciona. https://en.wikipedia.org/wiki/Software_testing
- **Ministry of Testing** — comunidad viva, con artículos nuevos cada semana. Es donde está la conversación real del oficio, no la teoría. <https://www.ministryoftesting.com/articles>

Un último consejo, y va en serio: **el temario oficial es gratis**. Descárgalo y ten el PDF abierto al lado de esta guía la semana antes del examen. Yo te expliqué las cosas para que se entiendan; el temario las dice con las palabras exactas con las que van a estar escritas las preguntas. Las dos cosas juntas valen mucho más que cualquiera de las dos por separado.

Grupo Mazi

Departamento de formación

GRUPO MAZI · DOCUMENTO ORIGINAL